

RTL Design Sherpa

RTL AMBA ACE Module Reference — Rev 0.92

October 2026

Table of Contents

1 ACE (AXI Coherency Extensions) Modules....8
1.1 Overview....9
1.1.1 What is here....9
1.1.2 Key Features....9
1.1.3 Module Categories....10
1.2 Functional Description....12
1.2.1 ACE adds to AXI4....12
1.2.2 Snoop channels....13
1.2.3 Ordering rules....13
1.2.4 CRRESP bit semantics....14
1.3 Timing....15
1.3.1 Throughput....15
1.3.2 Latency....15
1.4 Usage Example....16
1.4.1 ACE read master....16
1.4.2 Cache-side snoop responder....16
1.5 Design Notes....18
1.5.1 Buffer depth guidelines....18
1.5.2 Snoop monitor vs. front-side monitor....18
1.5.3 What is not here....18
1.6 Related Modules....19
1.7 References....20

1.8 Navigation....22
2 axi4ace_master_rd....23
2.1 Overview....24
2.2 Parameters....25
2.3 Ports....26
2.3.1 Module Declaration....26
2.3.2 Clock and Reset....28
2.3.3 Slave AXI Interface (Input Side - FUB)....28
2.3.4 Master AXI Interface (Output Side)....30
2.3.5 ACE Acknowledge....31
2.3.6 Status Interface....31
2.4 Functional Description....33
2.4.1 Block diagram....33
2.4.2 Dual Buffer Design....33
2.4.3 RACK Generation....33
2.4.4 Busy Signal Generation....34
2.5 ACE Notes....35
2.6 Related Modules....36
2.7 References....37
2.8 Navigation....38
3 axi4ace_master_wr....39
3.1 Overview....40
3.2 Parameters....41
3.3 Ports....42

3.3.1 Module Declaration....42
3.3.2 Clock and Reset....44
3.3.3 Slave AXI Interface (Input Side - FUB)....44
3.3.4 Master AXI Interface (Output Side)....46
3.3.5 ACE Acknowledge....48
3.3.6 Status Interface....48
3.4 Functional Description....49
3.4.1 Block diagram....49
3.4.2 Triple Buffer Design....49
3.4.3 WACK Generation....49
3.4.4 Busy Signal Generation....50
3.5 ACE Notes....51
3.6 Related Modules....52
3.7 References....53
3.8 Navigation....54
4 axi4ace_slave_rd....55
4.1 Overview....56
4.2 Parameters....57
4.3 Ports....58
4.3.1 Module Declaration....58
4.3.2 Clock and Reset....60
4.3.3 Slave AXI Interface (Input Side)....60
4.3.4 Master AXI Interface (Output Side - FUB)....62
4.3.5 ACE Acknowledge....63

4.3.6 Status Interface....64
4.4 Functional Description....65
4.4.1 Block diagram....65
4.4.2 Dual Buffer Design....65
4.4.3 RACK Forwarding....65
4.4.4 Busy Signal Generation....66
4.5 ACE Notes....67
4.6 Related Modules....68
4.7 References....69
4.8 Navigation....70
5 axi4ace_slave_wr....71
5.1 Overview....72
5.2 Parameters....73
5.3 Ports....74
5.3.1 Module Declaration....74
5.3.2 Clock and Reset....76
5.3.3 Slave AXI Interface (Input Side)....76
5.3.4 Master AXI Interface (Output Side - FUB)....78
5.3.5 ACE Acknowledge....80
5.3.6 Status Interface....81
5.4 Functional Description....82
5.4.1 Block diagram....82
5.4.2 Triple Buffer Design....82
5.4.3 Busy Signal Generation....82

5.5 ACE Notes....83
5.6 Related Modules....84
5.7 References....85
5.8 Navigation....86
6 axi4ace_snoop_master....87
6.1 Overview....88
6.2 Parameters....89
6.3 Ports....90
6.3.1 Module Declaration....90
6.3.2 Clock and Reset....91
6.3.3 CCU Interface (Input Side - FUB)....91
6.3.4 Cache Responder Interface (Output Side - AXI)....92
6.3.5 Status Interface....93
6.4 Functional Description....94
6.4.1 Block diagram....94
6.4.2 Triple Buffer Design....94
6.4.3 Busy Signal Generation....94
6.5 ACE Notes....95
6.6 Related Modules....96
6.7 References....97
6.8 Navigation....99
7 axi4ace_snoop_slave....100
7.1 Overview....101
7.2 Parameters....102

7.3 Ports....103
7.3.1 Module Declaration....103
7.3.2 Clock and Reset....104
7.3.3 Manager Interface (Input Side - AXI)....104
7.3.4 Cache FSM Interface (Output Side - FUB)....105
7.3.5 Status Interface....106
7.4 Functional Description....107
7.4.1 Block diagram....107
7.4.2 Triple Buffer Design....107
7.4.3 Busy Signal Generation....107
7.5 ACE Notes....108
7.6 Related Modules....109
7.7 References....110
7.8 Navigation....112

1 ACE (AXI Coherency Extensions) Modules

Location: `rtl/amba/ace/` **Status:** Production Ready

1.1 Overview

The ACE subsystem is the AMBA AXI Coherency Extensions transport layer for the `cache-ip` family (amber/jet caches, onyx CCU). ACE extends AXI4 with coherent transaction-type, shareability-domain and barrier fields on the address channels (`ARSNOOP[3:0]`/`AWSNOOP[2:0]`, `ARDOMAIN[1:0]`/`AWDOMAIN[1:0]`, `ARBAR[1:0]`/`AWBAR[1:0]`), a 4-bit `RRESP` on read data, and three additional snoop channels (AC/CR/CD) so that a coherency manager can broadcast snoops to peer caches and gather their responses.

These modules are **channel movers with skid buffers**, not protocol controllers. They carry the ACE-shaped port contract defined in `projects/components/cache-ip/References/AMBA_ACE_Interface_Definition.md`: the onyx D2 subset supports ReadShared, ReadUnique, CleanUnique, MakeUnique, WriteBack, and Evict; full ACE/ACE-Lite conformance is intentionally not claimed.

1.1.1 What is here

- Six base channel movers (base + `_monlite` only; no `_cg`, `_mon`, or stubs):
  - Front-side read/write masters and slaves (`axi4ace_master_rd/wr`, `axi4ace_slave_rd/wr`)
  - Snoop-side cache responder and CCU initiator (`axi4ace_snoop_slave/master`)
- Eight `_monlite` wrappers in `rtl/amba/ace/` that wrap the base movers with the lite-monitor discipline.
- One new ACE-aware monitor core in `rtl/amba/ocelus/axi4ace_snoop_monitor_lite.sv` that tracks snoops in AC-issue order because snoop channels have no ID.

1.1.2 Key Features

- **AXI4 + ACE payloads:** Full AXI4 address/data channels plus `AxSNOOP`, `AxDOMAIN`, `AxBAR` and the 4-bit ACE `RRESP` (IHI 0022H §D2.1). The optional `AWUNIQUE` (WriteEvict only) is not carried
- **Acknowledges:** Master read/write movers generate `RACK`/`WACK` one cycle after the transaction completes (no ordering semantics in the current subset); the slave movers take the manager's `RACK`/`WACK` and forward it to the backend
- **Snoop transport:** AC in, CR/CD out (cache side) or AC out, CR/CD in (CCU side)
- **Lite monitoring:** Front-side wrappers use `axi_monitor_lite`; snoop-side wrappers use `axi4ace_snoop_monitor_lite`

- **Skid-buffer elasticity:** Per-channel configurable depth, same `gaxi_skid_buffer` infrastructure as the rest of the AMBA library

1.1.3 Module Categories

1.1.3.1 Core Front-Side Movers

Module	Description	Documentation	Status
axi4ace_master_rd	ACE read master with `ARSNOOP`/`ARDOMAIN`/`ARBAR`, 4-bit `RRESP` and auto-pulsed `RACK`	axi4ace_master_rd.md	Documented
axi4ace_master_wr	ACE write master with `AWSNOOP`/`AWDOMAIN`/`AWBAR` and auto-pulsed `WACK`	axi4ace_master_wr.md	Documented
axi4ace_slave_rd	ACE read slave: `ARSNOOP`/`ARDOMAIN`/`ARBAR` and 4-bit `RRESP` pass-through, `RACK` forwarded	axi4ace_slave_rd.md	Documented
axi4ace_slave_wr	ACE write slave: `AWSNOOP`/`AWDOMAIN`/`AWBAR` pass-through, `WACK` forwarded	axi4ace_slave_wr.md	Documented

1.1.3.2 Snoop-Side Movers

Module	Description	Documentation	Status
axi4ace_snoop_slave	Cache-side snoop responder transport: AC in, CR/CD out	axi4ace_snoop_slave.md	Documented
axi4ace_snoop_master	CCU-side snoop initiator transport: AC out, CR/CD in	axi4ace_snoop_master.md	Documented

1.1.3.3 Monitor Wrappers

All eight wrappers are documented collectively in monitor/axi_monitor_lite_wrappers.md.

Wrapper	Core	Monitor core	Role
axi4ace_master_rd_monlite	axi4ace_master_rd	axi_monitor_lite	Front-side read master + lite monitor
axi4ace_master_wr_monlite	axi4ace_master_wr	axi_monitor_lite	Front-side write master + lite monitor
axi4ace_slave_rd_monlite	axi4ace_slave_rd	axi_monitor_lite	Front-side read slave + lite monitor
axi4ace_slave_wr_monlite	axi4ace_slave_wr	axi_monitor_lite	Front-side write slave + lite monitor
axi4ace_snoop_slave_monlite	axi4ace_snoop_slave	axi4ace_snoop_monitor_lite	Cache-side snoop responder + snoop monitor
axi4ace_snoop_master_monlite	axi4ace_snoop_master	axi4ace_snoop_monitor_lite	CCU-side snoop initiator + snoop monitor

The front-side `_monlite` wrappers use the same `axi_monitor_lite` taps as the AXI4/AXI5 families; the snoop-side wrappers use the new `axi4ace_snoop_monitor_lite` core because snoop channels have no transaction ID.

1.2 Functional Description

1.2.1 ACE adds to AXI4

On the front side, ACE adds these signals to the AXI4 channels (IHI 0022H §D2.1, Tables D2-1 to D2-3):

Channel	Field	Width	Meaning
AR	ARSNOOP	4	Read-side transaction type
AR	ARDOMAIN	2	Shareability domain: 00 Non-shareable, 01 Inner, 10 Outer, 11 System (Table D3-2)
AR	ARBAR	2	Barrier: 00 normal, 01 memory barrier, 10 normal ignoring barriers, 11 synchronization barrier (Table D3-5)
AW	AWSNOOP	3	Write-side transaction type
AW	AWDOMAIN	2	Shareability domain, as ARDOMAIN
AW	AWBAR	2	Barrier, as ARBAR
R	RRESP[3:2]	2	RRESP is 4 bits: [2] PassDirty, [3] IsShared (Table D3-14)

W and B are unchanged AXI4. The movers carry every field without decoding it; whether a given AxSNOOP/AxDOMAIN/AxBAR combination is legal (Tables D3-7 and D3-8) is the issuing manager's responsibility. AWUNIQUE, required only by a component that supports WriteEvict, is not carried. The front-side monitors see RRESP[1:0] only, so IsShared and PassDirty are never counted as errors.

The manager additionally drives the acknowledges (§D2.3):

Signal	Master movers	Slave movers	Meaning
RACK	Output, auto-pulsed	s_axi_rack in,	One-cycle

Signal	Master movers	Slave movers	Meaning
		forwarded to `fub_axi_rack`	pulse after the last read data beat
`WACK`	Output, auto-pulsed	`s_axi_wack` in, forwarded to `fub_axi_wack`	One-cycle pulse after the write response

The current subset gives these acknowledges no ordering semantics; the master movers auto-pulse them promptly and the slave movers forward them as wires.

1.2.2 Snoop channels

The snoop side uses three channels between the coherency manager (onyx) and each peer cache (amber/jet):

Channel	Dir (cache view)	Payload	Meaning
AC	In	`ACADDR`, `ACSNOOP[3:0]`, `ACPROT[2:0]`	Snoop address/command
CR	Out	`CRRESP[4:0]`	Snoop response
CD	Out	`CDDATA`, `CDLAST`	Snoop data

The cache-side mover is `axi4ace_snoop_slave`; the manager-side mover is `axi4ace_snoop_master`. Both are pure transports: they buffer the channels and present the same payload on the FUB side as on the AXI side.

1.2.3 Ordering rules

ACE requires:

- CR and CD only after the AC handshake
- CR responses in the same order as AC addresses
- CD data in the same order as AC addresses
- Multiple outstanding snoops are allowed

The `cache-ip` family convention is stricter: a cache completes all CD beats for a snoop before asserting its CR, so onyx sees one fully-closed response at a time. That convention is enforced by the cache adapter, not by these transport modules.

1.2.4 CRRESP bit semantics

Bit	Name	Meaning when set
[0]	DataTransfer	Responder will drive data on CD
[1]	Error	Responder’s lookup had an error
[2]	PassDirty	Dirty responsibility moves with the data
[3]	IsShared	Another master may hold a copy
[4]	WasUnique	Line was unique in the responder (IHI 0022)

1.3 Timing

1.3.1 Throughput

Configuration	Address CH	Data CH	Notes
Single transaction	1 txn/cycle	1 beat/cycle	When buffers not stalled
Burst (len=16)	1 txn/16 cycles	1 beat/cycle	Sustained
Multiple outstanding	Up to table depth	1 beat/cycle	Out-of-order on front side, in-order on snoop side

1.3.2 Latency

Path	Cycles	Notes
AR → R (single)	2-3	Minimum read latency
AW → B (single)	2-3	Minimum write latency
AC → CR (single)	2-3	Snoop response latency
Buffer overhead	+1 per channel	Skid buffer latency

1.4 Usage Example

1.4.1 ACE read master

```
axi4ace_master_rd #(
    .SKID_DEPTH_AR(2),
    .SKID_DEPTH_R(4),
    .AXI_ID_WIDTH(8),
    .AXI_ADDR_WIDTH(32),
    .AXI_DATA_WIDTH(64)
) u_ace_rd_master (
    .aclk           (axi_clk),
    .aresetn        (axi_resetn),
    // FUB side (upstream requestor)
    .fub_axi_arid   (cpu_arid),
    .fub_axi_araddr (cpu_araddr),
    .fub_axi_arlen  (cpu_arlen),
    .fub_axi_arsnoop(cpu_arsnoop),   // ACE read transaction type
    // ...
    // Master side (downstream manager/CCU)
    .m_axi_arid     (m_arid),
    .m_axi_araddr   (m_araddr),
    .m_axi_arsnoop  (m_arsnoop),
    // ...
    .m_axi_rack     (m_rack),        // auto-pulsed by the module
    .busy           (rd_busy)
);
```

1.4.2 Cache-side snoop responder

```
axi4ace_snoop_slave #(
    .SKID_DEPTH_AC(2),
    .SKID_DEPTH_CR(4),
    .SKID_DEPTH_CD(4),
    .ADDR_WIDTH(32),
    .DATA_WIDTH(64)
) u_snoop_slave (
    .aclk          (axi_clk),
    .aresetn       (axi_resetn),
    // Manager/CCU side
    .m_axi_acaddr  (snp_acaddr),
    .m_axi_acsnoop (snp_acsnoop),
    .m_axi_acprot  (snp_acprot),
    .m_axi_acvalid (snp_acvalid),
    .m_axi_acready (snp_acready),
    .m_axi_crresp  (snp_crresp),
    .m_axi_crvalid (snp_crvalid),
    .m_axi_crready (snp_crready),
    .m_axi_cddata  (snp_cddata),
```

```
    .m_axi_cdlast  (snp_cdlast),
    .m_axi_cdvalid (snp_cdvalid),
    .m_axi_cdready (snp_cdready),
    // Cache FSM side
    .fub_acaddr    (fub_acaddr),
    .fub_acsnoop   (fub_acsnoop),
    .fub_acprot    (fub_acprot),
    .fub_acvalid   (fub_acvalid),
    .fub_acready   (fub_acready),
    .fub_crresp    (fub_crresp),
    .fub_crvalid   (fub_crvalid),
    .fub_crready   (fub_crready),
    .fub_cddata    (fub_cddata),
    .fub_cdlast    (fub_cdlast),
    .fub_cdvalid   (fub_cdvalid),
    .fub_cdready   (fub_cdready),
    .busy          (snp_busy)
);
```

1.5 Design Notes

1.5.1 Buffer depth guidelines

SKID_DEPTH_* is an entry count, not a log2 exponent. Legal values are 2..8 inclusive (any integer, odd depths are legal). Values above 8 fail elaboration; place a gaxi_fifo_sync stage ahead of the module for deeper elasticity.

1.5.2 Snoop monitor vs. front-side monitor

The front-side ACE wrappers reuse axi_monitor_lite, which tracks AXI transactions by ID. The snoop-side wrappers use axi4ace_snoop_monitor_lite because snoop channels have no ID: the monitor keeps a circular table of outstanding snoops in AC-issue order and attributes each CR/CDLAST to the oldest entry.

1.5.3 What is not here

- No _cg variants: clock gating is not provided for ACE in this release
- No _mon variants: only the lite-monitor wrappers exist
- No stubs: these are production channel movers
- ACE-Lite is not separately implemented; the cache-ip contract is full ACE-shaped on the pins documented here

1.6 Related Modules

- axi4ace_snoop_monitor_lite — ACE snoop-channel monitor core
- axi_monitor_lite_wrappers — the eight `_monlite` wrappers
- AXI4 modules — the base protocol these modules extend
- AMBA ACE Interface Definition — the in-repo port contract

1.7 References

- Arm. *AMBA AXI and ACE Protocol Specification*, ARM IHI 0022H.c, Jan. 2021, Part D “AMBA ACE and ACE-Lite Protocol Specification”. developer.arm.com/documentation/ihi0022/h — ch. D2 “Signal Descriptions”, ch. D3 “Channel Signaling” (§D3.3 `RACK`, §D3.5 `WACK`, §D3.6-§D3.9 snoop channels and their dependencies), ch. D5 “Snoop Transactions”. Issues J and later remove ACE from IHI 0022, so Issue H.c is the last to define it. Cited, not mirrored.
- Carloni, L.P., McMillan, K.L., Sangiovanni-Vincentelli, A.L. “Theory of Latency-Insensitive Design.” *IEEE Trans. Computer-Aided Design of Integrated Circuits and Systems*, 20(9), Sept. 2001. DOI: 10.1109/43.945302 — the valid/ready channel discipline behind the `gaxi_skid_buffer` queues: a buffered stage may stall or add latency without changing what the endpoints compute.
- Papamarcos, M.S., Patel, J.H. “A Low-Overhead Coherence Solution for Multiprocessors with Private Cache Memories.” *Proc. 11th Int. Symp. Computer Architecture (ISCA)*, 1984, pp. 348-354. DOI: 10.1145/800015.808204 — the snoopy write-back protocol with an exclusive-clean state (MESI) that the snoop channels serve; background only, since this family holds no coherence state.
- Sweazey, P., Smith, A.J. “A Class of Compatible Cache Consistency Protocols and their Support by the IEEE Futurebus.” *Proc. 13th ISCA*, 1986; *ACM SIGARCH Computer Architecture News*, 14(2), May 1986, pp. 414-423. DOI: 10.1145/17356.17404 — the MOESI state class (the Owned state, `PassDirty` ownership transfer); background only.
- Nagarajan, V., Sorin, D.J., Hill, M.D., Wood, D.A. *A Primer on Memory Consistency and Cache Coherence*, 2nd ed. Synthesis Lectures on Computer Architecture, Morgan & Claypool, 2020. DOI: 10.1007/978-3-031-01764-3 — ch. 7 “Snooping Coherence Protocols” (§7.3 Exclusive state, §7.4 Owned state, §7.5 non-atomic bus): what a snoop request and response mean to the caches at each end.
- Maaref, M. *Architecting and Building High-Speed SoCs*, Packt, 2022, ch. 3 “Basic and Advanced On-Chip Busses and Interconnects”, sections “ARM AMBA interconnect protocols suite” (ACE) and “Data sharing and coherency challenges” — textbook treatment (secondary source).

- Handy, J. *The Cache Memory Book*, 2nd ed., 1998, ch. 4 “Maintaining Coherency in Cached Systems”, §4.3.1 “The MESI Protocol” and §4.3.3 “The MOESI Protocol” — textbook treatment (secondary source).
- Hennessy, J.L., Patterson, D.A. *Computer Architecture: A Quantitative Approach*, 6th ed., Morgan Kaufmann, 2019, ch. 5 “Thread-Level Parallelism”, §5.2 “Centralized Shared-Memory Architectures” — snooping coherence and the MESI/MOESI extensions (secondary source).
- Chipressian. *Crack the Hardware Interview - from RTL Designers' Perspective*, Part 1 ch. 6 “Cache Coherency” — snooping coherence (secondary source).
- `projects/components/cache-ip/References/AMBA_ACE_Interface_Definition.md` — working interface definition for this family
- Indexed in `rtl/amba/references/README.md` (local PDF: `LatencyInsensitiveDesign__Carloni2001.pdf`, `MoesiCoherenceClass__Sweazey1986.pdf`, `projects/components/cache-ip/References/Papamarcos_Patel_1984_ISCA_Snoopy_Coherence.pdf`, `projects/components/cache-ip/References/Sorin_Hill_Wood_2020_Primer_Memory_Consistency_Cache_Coherence_2ed.pdf`)

Last Updated: 2026-10-12

1.8 Navigation

- ← Back to rtl-amba Index
- ← Back to Main Documentation Index

2 axi4ace_master_rd

An AXI4 + ACE read master module that buffers read address and data channels while carrying the ACE `ARSNOOP[3:0]`, `ARDOMAIN[1:0]` and `ARBAR[1:0]` address fields and the 4-bit ACE `RRESP`, and auto-pulsing the `RACK` acknowledge.

2.1 Overview

The axi4ace_master_rd module is the ACE extension of axi4_master_rd. It provides the same dual skid-buffer architecture for the AR and R channels, adds ARSNOOP[3:0], ARDOMAIN[1:0] and ARBAR[1:0] to the address-channel payload, widens RRESP to the 4 ACE bits on the data-channel payload, and generates a one-cycle RACK pulse after the last read data beat. It is a transport module: it does not interpret snoop encodings or maintain coherency state.

2.2 Parameters

Parameter	Type	Default	Description
SKID_DEPTH_AR	int	2	Address channel skid buffer depth in entries (2..8 inclusive)
SKID_DEPTH_R	int	4	Read data channel skid buffer depth in entries (2..8 inclusive)
AXI_ID_WIDTH	int	8	AXI transaction ID width
AXI_ADDR_WIDTH	int	32	AXI address bus width
AXI_DATA_WIDTH	int	32	AXI data bus width
AXI_USER_WIDTH	int	1	AXI user signal width
AW	int	AXI_ADDR_WIDTH	Short alias for address width
DW	int	AXI_DATA_WIDTH	Short alias for data width
IW	int	AXI_ID_WIDTH	Short alias for ID width
UW	int	AXI_USER_WIDTH	Short alias for user width
ARSize	int	IW+AW+8+3+2+1+4+3+4+4+UW+4+2+2	Packed AR skid payload width
RSize	int	IW+DW+4+1+UW	Packed R skid payload width

2.3 Ports

2.3.1 Module Declaration

```
module axi4ace_master_rd #(
    parameter int SKID_DEPTH_AR     = 2,
    parameter int SKID_DEPTH_R      = 4,

    parameter int AXI_ID_WIDTH      = 8,
    parameter int AXI_ADDR_WIDTH    = 32,
    parameter int AXI_DATA_WIDTH    = 32,
    parameter int AXI_USER_WIDTH    = 1,
    // Short and calculated params
    parameter int AW       = AXI_ADDR_WIDTH,
    parameter int DW       = AXI_DATA_WIDTH,
    parameter int IW       = AXI_ID_WIDTH,
    parameter int UW       = AXI_USER_WIDTH,
    parameter int ARSize   = IW+AW+8+3+2+1+4+3+4+4+UW+4+2+2,
    parameter int RSize    = IW+DW+4+1+UW
)
(
    // Global Clock and Reset
    input  logic                        aclk,
    input  logic                        aresetn,

    // Slave AXI Interface (Input Side)
    // Read address channel (AR)
    input  logic [IW-1:0]               fub_axi_arid,
    input  logic [AW-1:0]               fub_axi_araddr,
    input  logic [7:0]                  fub_axi_arlen,
    input  logic [2:0]                  fub_axi_arsize,
    input  logic [1:0]                  fub_axi_arburst,
    input  logic                        fub_axi_arlock,
    input  logic [3:0]                  fub_axi_arcache,
    input  logic [2:0]                  fub_axi_arprot,
    input  logic [3:0]                  fub_axi_arqos,
    input  logic [3:0]                  fub_axi_arregion,
    input  logic [UW-1:0]               fub_axi_aruser,
    input  logic [3:0]                  fub_axi_arsnoop,
    input  logic [1:0]                  fub_axi_ardomain,
    input  logic [1:0]                  fub_axi_arbar,
    input  logic                        fub_axi_arvalid,
    output logic                        fub_axi_arready,

    // Read data channel (R)
    output logic [IW-1:0]               fub_axi_rid,
```

```
    output logic [DW-1:0]               fub_axi_rdata,
    output logic [3:0]                  fub_axi_rresp,       //
[3]=IsShared [2]=PassDirty (IHI 0022H D2.1.3)
    output logic                        fub_axi_rlast,
    output logic [UW-1:0]               fub_axi_ruser,
    output logic                        fub_axi_rvalid,
    input  logic                        fub_axi_rready,

    // Master AXI Interface (Output Side)
    // Read address channel (AR)
    output logic [IW-1:0]               m_axi_arid,
    output logic [AW-1:0]               m_axi_araddr,
    output logic [7:0]                  m_axi_arlen,
    output logic [2:0]                  m_axi_arsize,
    output logic [1:0]                  m_axi_arburst,
    output logic                        m_axi_arlock,
    output logic [3:0]                  m_axi_arcache,
    output logic [2:0]                  m_axi_arprot,
    output logic [3:0]                  m_axi_arqos,
    output logic [3:0]                  m_axi_arregion,
    output logic [UW-1:0]               m_axi_aruser,
    output logic [3:0]                  m_axi_arsnoop,
    output logic [1:0]                  m_axi_ardomain,
    output logic [1:0]                  m_axi_arbar,
    output logic                        m_axi_arvalid,
    input  logic                        m_axi_arready,

    // Read data channel (R)
    input  logic [IW-1:0]               m_axi_rid,
    input  logic [DW-1:0]               m_axi_rdata,
    input  logic [3:0]                  m_axi_rresp,       //
[3]=IsShared [2]=PassDirty (IHI 0022H D2.1.3)
    input  logic                        m_axi_rlast,
    input  logic [UW-1:0]               m_axi_ruser,
    input  logic                        m_axi_rvalid,
    output logic                        m_axi_rready,

    // ACE read acknowledge
    output logic                        m_axi_rack,

    // Status outputs for clock gating
    output logic                        busy
);
```

2.3.2 Clock and Reset

Port	Width	Direction	Description
aclk	1	Input	AXI clock
aresetn	1	Input	AXI active-low reset

2.3.3 Slave AXI Interface (Input Side - FUB)

2.3.3.1 Read Address Channel (AR)

Port	Width	Direction	Description
fub_axi_arid	AXI_ID_WIDTH	Input	Read address ID
fub_axi_araddr	AXI_ADDR_WIDTH	Input	Read address
fub_axi_arlen	8	Input	Burst length, AXI-encoded: beats - 1
fub_axi_arsize	3	Input	Transfer size (bytes per beat)
fub_axi_arburst	2	Input	Burst type (FIXED, INCR, WRAP)
fub_axi_arlock	1	Input	Lock type (atomic access)
fub_axi_arcache	4	Input	Cache attributes
fub_axi_arprot	3	Input	Protection attributes
fub_axi_arqos	4	Input	Quality of Service
fub_axi_arregion	4	Input	Region identifier
fub_axi_aruser	AXI_USER_WIDTH	Input	User-defined signals
fub_axi_arsnoop	4	Input	ACE read transaction type
fub_axi_ar	2	Input	ACE shareability domain

Port	Width	Direction	Description
domain			(ARDOMAIN): 00 Non-shareable, 01 Inner, 10 Outer, 11 System
fub_axi_arbar	2	Input	ACE barrier type (ARBAR): 00 normal, 01 memory barrier, 10 normal ignoring barriers, 11 synchronization barrier
fub_axi_arvalid	1	Input	Read address valid
fub_axi_arready	1	Output	Read address ready

2.3.3.2 Read Data Channel (R)

Port	Width	Direction	Description
fub_axi_rid	AXI_ID_WIDTH	Output	Read data ID
fub_axi_rdata	AXI_DATA_WIDTH	Output	Read data
fub_axi_rresp	4	Output	ACE read response: [1:0] OKAY/EXOKAY/SLVERR/DECERR, [2] PassDirty, [3] IsShared
fub_axi_rlast	1	Output	Last read transfer in burst
fub_axi_ruser	AXI_USER_WIDTH	Output	User-defined signals
fub_axi_rvalid	1	Output	Read data valid
fub_axi_rready	1	Input	Read data ready

2.3.4 Master AXI Interface (Output Side)

2.3.4.1 Read Address Channel (AR)

Port	Width	Direction	Description
m_axi_arid	AXI_ID_WIDTH	Output	Read address ID
m_axi_araddr	AXI_ADDR_WIDTH	Output	Read address
m_axi_arlen	8	Output	Burst length, AXI-encoded: beats - 1
m_axi_arsize	3	Output	Transfer size (bytes per beat)
m_axi_arburst	2	Output	Burst type (FIXED, INCR, WRAP)
m_axi_arlock	1	Output	Lock type (atomic access)
m_axi_arcache	4	Output	Cache attributes
m_axi_arprot	3	Output	Protection attributes
m_axi_arqos	4	Output	Quality of Service
m_axi_arregion	4	Output	Region identifier
m_axi_aruser	AXI_USER_WIDTH	Output	User-defined signals
m_axi_arsnoop	4	Output	ACE read transaction type
m_axi_ardomain	2	Output	ACE shareability domain (ARDOMAIN): 00 Non-shareable, 01 Inner, 10 Outer, 11 System
m_axi_arbar	2	Output	ACE barrier type (ARBAR): 00 normal, 01 memory barrier, 10 normal

Port	Width	Direction	Description
			ignoring barriers, 11 synchronization barrier
m_axi_arvalid	1	Output	Read address valid
m_axi_arready	1	Input	Read address ready

2.3.4.2 Read Data Channel (R)

Port	Width	Direction	Description
m_axi_rid	AXI_ID_WIDTH	Input	Read data ID
m_axi_rdata	AXI_DATA_WIDTH	Input	Read data
m_axi_rresp	4	Input	ACE read response: [1:0] OKAY/EXOKAY/SLVERR/DECERR, [2] PassDirty, [3] IsShared
m_axi_rlast	1	Input	Last read transfer in burst
m_axi_ruser	AXI_USER_WIDTH	Input	User-defined signals
m_axi_rvalid	1	Input	Read data valid
m_axi_rready	1	Output	Read data ready

2.3.5 ACE Acknowledge

Port	Width	Direction	Description
m_axi_rack	1	Output	One-cycle pulse one cycle after the last read data beat handshake

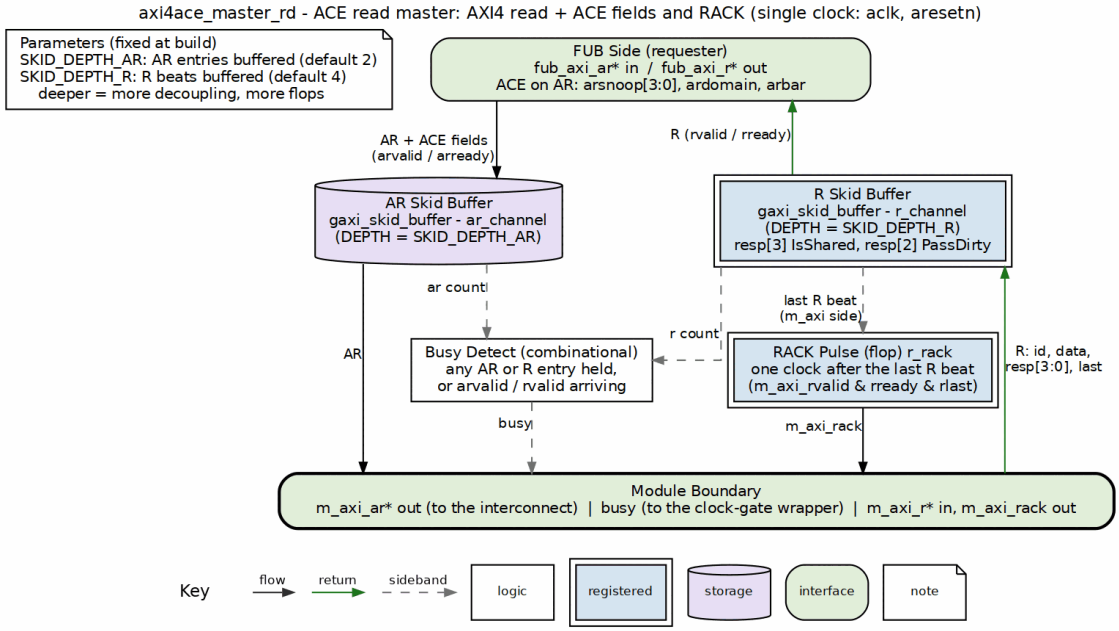
2.3.6 Status Interface

Port	Width	Direction	Description
busy	1	Output	Module

Port	Width	Direction	Description
			activity indicator for clock gating

2.4 Functional Description

2.4.1 Block diagram



axi4ace_master_rd block diagram

The axi4 read master's layout (see `axi4_master_rd`) plus what ACE adds. AR drops down the left through its skid buffer, carrying `arsnoop`, `ardomain` and `arbar` with the AXI4 fields. R climbs the right through its skid buffer with the 4-bit `rresp`, where bit 3 is IsShared and bit 2 is PassDirty. The RACK flop pulses `m_axi_rack` for one clock after the last R beat is accepted on the `m_axi` side. Busy detect feeds the clock-gate wrapper.

2.4.2 Dual Buffer Design

The module employs the same dual skid buffer architecture as `axi4_master_rd`:

1. **AR Channel Buffer**: Buffers read address transactions, including `ARSNOOP`, `ARDOMAIN` and `ARBAR`
2. **R Channel Buffer**: Buffers read data responses, including all four `RRESP` bits

2.4.3 RACK Generation

The ACE read acknowledge is auto-pulsed by the module:

```
assign m_axi_rack = r_rack;
// r_rack <= m_axi_rvalid && m_axi_rready && m_axi_rlast;
```

A one-cycle pulse is emitted one cycle after the last read data beat handshake. The current subset does not rely on RACK for ordering; downstream consumers that need ordering semantics must implement them externally.

2.4.4 Busy Signal Generation

```
assign busy = (int_ar_count > 0) || (int_r_count > 0) ||
                fub_axi_arvalid || m_axi_rvalid;
```

2.5 ACE Notes

- `ARSNOOP[3:0]`, `ARDOMAIN[1:0]` and `ARBAR[1:0]` are straight payload pass-throughs; the module does not decode them or check that the combination is one Table D3-7 permits
- `RRESP[3:2]` (IsShared, PassDirty) rides the R skid with `RRESP[1:0]`; the module does not interpret it. A consumer that ignores it must make sure its manager never sends a response it cannot honour (see Table D3-15)
- `RACK` is generated locally; do not drive `m_axi_rack` from outside
- IDs (`ARID`/`RID`) are carried end-to-end; the module performs no ID matching or reordering
- The module is part of the ACE-shaped port contract for the `cache-ip` family; see the contract doc for transaction-type semantics

2.6 Related Modules

- **axi4ace_master_rd_monlite**: Lite-monitor wrapper for this core
- **axi4ace_slave_rd**: Complementary ACE read slave
- **axi4_master_rd**: AXI4-only base module
- **gaxi_skid_buffer**: Underlying buffer infrastructure

2.7 References

- Arm. *AMBA AXI and ACE Protocol Specification*, ARM IHI 0022H.c, Jan. 2021, Part D “AMBA ACE and ACE-Lite Protocol Specification”. developer.arm.com/documentation/ihi0022/h – §D2.1 “Changes to existing AXI channels” (`ARSNOOP`, `ARDOMAIN`, `ARBAR`, the 4-bit `RRESP`), §D3.1 Tables D3-2, D3-5 and D3-7 (domain and barrier encodings and their permitted combinations), §D3.2 read data channel signaling (Table D3-14, `RRESP[3:2]` = IsShared, PassDirty), §D3.3 “Read acknowledge signaling” (`RACK`). The base valid/ready handshake is Part A §A3.2. Citation only.
- Carloni, L.P., McMillan, K.L., Sangiovanni-Vincentelli, A.L. “Theory of Latency-Insensitive Design.” *IEEE Trans. Computer-Aided Design of Integrated Circuits and Systems*, 20(9), Sept. 2001. DOI: 10.1109/43.945302 – the valid/ready channel discipline behind the `gaxi_skid_buffer` queues: a buffered stage may stall or add latency without changing what the endpoints compute.
- Maaref, M. *Architecting and Building High-Speed SoCs*, Packt, 2022, ch. 3 “Basic and Advanced On-Chip Busses and Interconnects”, sections “ARM AMBA interconnect protocols suite” (ACE) and “Data sharing and coherency challenges” – textbook treatment (secondary source).
- Indexed in `rtl/amba/references/README.md` (local PDF: `LatencyInsensitiveDesign__Carloni2001.pdf`)

2.8 Navigation

- ← Back to ACE Index
- ← Back to rtl-amba Index
- ← Back to Main Documentation Index

3 axi4ace_master_wr

An AXI4 + ACE write master module that buffers AW, W, and B channels while carrying the ACE `AWSNOOP[2:0]`, `AWDOMAIN[1:0]` and `AWBAR[1:0]` address fields and auto-pulsing the `WACK` acknowledge.

3.1 Overview

The `axi4ace_master_wr` module is the ACE extension of `axi4_master_wr`. It provides skid-buffer elasticity for the AW, W, and B channels, adds `AWSNOOP[2:0]`, `AWDOMAIN[1:0]` and `AWBAR[1:0]` to the write address payload, and generates a one-cycle `WACK` pulse after the write response handshake. It is a transport module and does not interpret snoop encodings.

3.2 Parameters

Parameter	Type	Default	Description
SKID_DEPTH_AW	int	2	Write address channel skid buffer depth in entries (2..8 inclusive)
SKID_DEPTH_W	int	4	Write data channel skid buffer depth in entries (2..8 inclusive)
SKID_DEPTH_B	int	2	Write response channel skid buffer depth in entries (2..8 inclusive)
AXI_ID_WIDTH	int	8	AXI transaction ID width
AXI_ADDR_WIDTH	int	32	AXI address bus width
AXI_DATA_WIDTH	int	32	AXI data bus width
AXI_USER_WIDTH	int	1	AXI user signal width
AXI_WSTRB_WIDTH	int	AXI_DATA_WIDTH/8	AXI write strobe width (calculated)
AW	int	AXI_ADDR_WIDTH	Short alias for address width
DW	int	AXI_DATA_WIDTH	Short alias for data width
IW	int	AXI_ID_WIDTH	Short alias for ID width
SW	int	AXI_WSTRB_WIDTH	Short alias for strobe width
UW	int	AXI_USER_WIDTH	Short alias for user width
AWSize	int	IW+AW+8+3+2+1+4+3+4+4+UW+3+2+2	Packed AW skid payload width
WSize	int	DW+SW+1+UW	Packed W skid payload width
BSize	int	IW+2+UW	Packed B skid payload width

3.3 Ports

3.3.1 Module Declaration

```
module axi4ace_master_wr #(
    parameter int SKID_DEPTH_AW     = 2,
    parameter int SKID_DEPTH_W      = 4,
    parameter int SKID_DEPTH_B      = 2,

    parameter int AXI_ID_WIDTH      = 8,
    parameter int AXI_ADDR_WIDTH    = 32,
    parameter int AXI_DATA_WIDTH    = 32,
    parameter int AXI_USER_WIDTH    = 1,
    parameter int AXI_WSTRB_WIDTH   = AXI_DATA_WIDTH / 8,
    // Short and calculated params
    parameter int AW       = AXI_ADDR_WIDTH,
    parameter int DW       = AXI_DATA_WIDTH,
    parameter int IW       = AXI_ID_WIDTH,
    parameter int SW       = AXI_WSTRB_WIDTH,
    parameter int UW       = AXI_USER_WIDTH,
    parameter int AWSize   = IW+AW+8+3+2+1+4+3+4+4+UW+3+2+2,
    parameter int WSize    = DW+SW+1+UW,
    parameter int BSize    = IW+2+UW
)
(
    // Global Clock and Reset
    input  logic                        aclk,
    input  logic                        aresetn,

    // Slave AXI Interface (Input Side)
    // Write address channel (AW)
    input  logic [IW-1:0]               fub_axi_awid,
    input  logic [AW-1:0]               fub_axi_awaddr,
    input  logic [7:0]                  fub_axi_awlen,
    input  logic [2:0]                  fub_axi_awsize,
    input  logic [1:0]                  fub_axi_awburst,
    input  logic                        fub_axi_awlock,
    input  logic [3:0]                  fub_axi_awcache,
    input  logic [2:0]                  fub_axi_awprot,
    input  logic [3:0]                  fub_axi_awqos,
    input  logic [3:0]                  fub_axi_awregion,
    input  logic [UW-1:0]               fub_axi_awuser,
    input  logic [2:0]                  fub_axi_awsnoop,
    input  logic [1:0]                  fub_axi_awdomain,
    input  logic [1:0]                  fub_axi_awbar,
    input  logic                        fub_axi_awvalid,
    output logic                        fub_axi_awready,
```

```
    // Write data channel (W)
    input  logic [DW-1:0]               fub_axi_wdata,
    input  logic [SW-1:0]               fub_axi_wstrb,
    input  logic                        fub_axi_wlast,
    input  logic [UW-1:0]               fub_axi_wuser,
    input  logic                        fub_axi_wvalid,
    output logic                        fub_axi_wready,

    // Write response channel (B)
    output logic [IW-1:0]               fub_axi_bid,
    output logic [1:0]                  fub_axi_bresp,
    output logic [UW-1:0]               fub_axi_buser,
    output logic                        fub_axi_bvalid,
    input  logic                        fub_axi_bready,

    // Master AXI Interface (Output Side)
    // Write address channel (AW)
    output logic [IW-1:0]               m_axi_awid,
    output logic [AW-1:0]               m_axi_awaddr,
    output logic [7:0]                  m_axi_awlen,
    output logic [2:0]                  m_axi_awsize,
    output logic [1:0]                  m_axi_awburst,
    output logic                        m_axi_awlock,
    output logic [3:0]                  m_axi_awcache,
    output logic [2:0]                  m_axi_awprot,
    output logic [3:0]                  m_axi_awqos,
    output logic [3:0]                  m_axi_awregion,
    output logic [UW-1:0]               m_axi_awuser,
    output logic [2:0]                  m_axi_awsnoop,
    output logic [1:0]                  m_axi_awdomain,
    output logic [1:0]                  m_axi_awbar,
    output logic                        m_axi_awvalid,
    input  logic                        m_axi_awready,

    // Write data channel (W)
    output logic [DW-1:0]               m_axi_wdata,
    output logic [SW-1:0]               m_axi_wstrb,
    output logic                        m_axi_wlast,
    output logic [UW-1:0]               m_axi_wuser,
    output logic                        m_axi_wvalid,
    input  logic                        m_axi_wready,

    // Write response channel (B)
    input  logic [IW-1:0]               m_axi_bid,
    input  logic [1:0]                  m_axi_bresp,
    input  logic [UW-1:0]               m_axi_buser,
```

```
    input  logic                        m_axi_bvalid,
    output logic                        m_axi_bready,

    // ACE write acknowledge
    output logic                        m_axi_wack,

    // Status outputs for clock gating
    output logic                        busy
);
```

3.3.2 Clock and Reset

Port	Width	Direction	Description
aclk	1	Input	AXI clock
aresetn	1	Input	AXI active-low reset

3.3.3 Slave AXI Interface (Input Side - FUB)

3.3.3.1 Write Address Channel (AW)

Port	Width	Direction	Description
fub_axi_awid	AXI_ID_WIDTH	Input	Write address ID
fub_axi_awaddr	AXI_ADDR_WIDTH	Input	Write address
fub_axi_awlen	8	Input	Burst length, AXI-encoded: beats - 1
fub_axi_awsize	3	Input	Transfer size (bytes per beat)
fub_axi_awburst	2	Input	Burst type (FIXED, INCR, WRAP)
fub_axi_awlock	1	Input	Lock type (atomic access)
fub_axi_awcache	4	Input	Cache attributes
fub_axi_awprot	3	Input	Protection attributes
fub_axi_awqos	4	Input	Quality of Service

Port	Width	Direction	Description
fub_axi_awregion	4	Input	Region identifier
fub_axi_awuser	AXI_USER_WIDTH	Input	User-defined signals
fub_axi_awsnoop	3	Input	ACE write transaction type
fub_axi_awdomain	2	Input	ACE shareability domain (AWDOMAIN): 00 Non-shareable, 01 Inner, 10 Outer, 11 System
fub_axi_awbar	2	Input	ACE barrier type (AWBAR): 00 normal, 01 memory barrier, 10 normal ignoring barriers, 11 synchronization barrier
fub_axi_awvalid	1	Input	Write address valid
fub_axi_awready	1	Output	Write address ready

3.3.3.2 Write Data Channel (W)

Port	Width	Direction	Description
fub_axi_wdata	AXI_DATA_WIDTH	Input	Write data
fub_axi_wstrb	AXI_WSTRB_WIDTH	Input	Write strobes
fub_axi_wlast	1	Input	Last write transfer in burst
fub_axi_wuser	AXI_USER_WIDTH	Input	User-defined signals
fub_axi_wvalid	1	Input	Write data valid

Port	Width	Direction	Description
fub_axi_wready	1	Output	Write data ready

3.3.3.3 Write Response Channel (B)

Port	Width	Direction	Description
fub_axi_bid	AXI_ID_WIDTH	Output	Write response ID
fub_axi_bresp	2	Output	Write response (OKAY, EXOKAY, SLVERR, DECERR)
fub_axi_buser	AXI_USER_WIDTH	Output	User-defined signals
fub_axi_bvalid	1	Output	Write response valid
fub_axi_bready	1	Input	Write response ready

3.3.4 Master AXI Interface (Output Side)

3.3.4.1 Write Address Channel (AW)

Port	Width	Direction	Description
m_axi_awid	AXI_ID_WIDTH	Output	Write address ID
m_axi_awaddr	AXI_ADDR_WIDTH	Output	Write address
m_axi_awlen	8	Output	Burst length, AXI-encoded: beats - 1
m_axi_awsize	3	Output	Transfer size (bytes per beat)
m_axi_awburst	2	Output	Burst type (FIXED, INCR, WRAP)
m_axi_awlock	1	Output	Lock type (atomic access)
m_axi_awcache	4	Output	Cache attributes

Port	Width	Direction	Description
m_axi_awprot	3	Output	Protection attributes
m_axi_awqos	4	Output	Quality of Service
m_axi_awregion	4	Output	Region identifier
m_axi_awuser	AXI_USER_WIDTH	Output	User-defined signals
m_axi_awsnoop	3	Output	ACE write transaction type
m_axi_awdomain	2	Output	ACE shareability domain (AWDOMAIN): 00 Non-shareable, 01 Inner, 10 Outer, 11 System
m_axi_awbar	2	Output	ACE barrier type (AWBAR): 00 normal, 01 memory barrier, 10 normal ignoring barriers, 11 synchronization barrier
m_axi_awvalid	1	Output	Write address valid
m_axi_awready	1	Input	Write address ready

3.3.4.2 Write Data Channel (W)

Port	Width	Direction	Description
m_axi_wdata	AXI_DATA_WIDTH	Output	Write data
m_axi_wstrb	AXI_WSTRB_WIDTH	Output	Write strobes
m_axi_wlast	1	Output	Last write transfer in burst

Port	Width	Direction	Description
m_axi_wuser	AXI_USER_WIDTH	Output	User-defined signals
m_axi_wvalid	1	Output	Write data valid
m_axi_wready	1	Input	Write data ready

3.3.4.3 Write Response Channel (B)

Port	Width	Direction	Description
m_axi_bid	AXI_ID_WIDTH	Input	Write response ID
m_axi_bresp	2	Input	Write response (OKAY, EXOKAY, SLVERR, DECERR)
m_axi_buser	AXI_USER_WIDTH	Input	User-defined signals
m_axi_bvalid	1	Input	Write response valid
m_axi_bready	1	Output	Write response ready

3.3.5 ACE Acknowledge

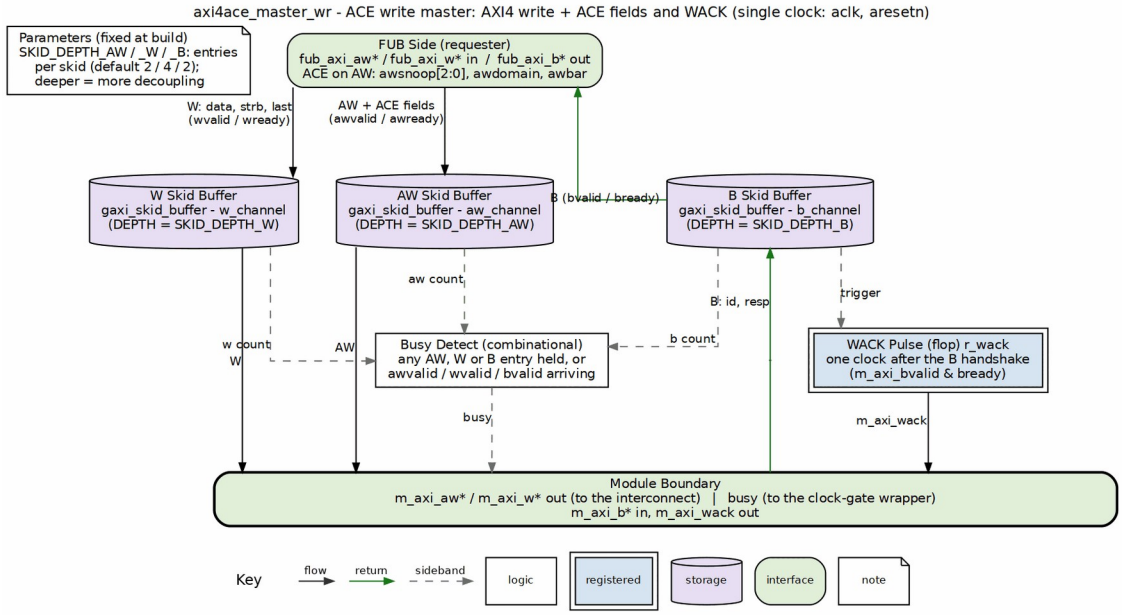
Port	Width	Direction	Description
m_axi_wack	1	Output	One-cycle pulse one cycle after the write response handshake

3.3.6 Status Interface

Port	Width	Direction	Description
busy	1	Output	Module activity indicator for clock gating

3.4 Functional Description

3.4.1 Block diagram



axi4ace_master_wr block diagram

The axi4 write master's layout (see `axi4_master_wr`) plus what ACE adds. W drops down the left lane and AW down the center, carrying `awsnoop`, `awdomain` and `awbar`. B climbs the right lane. The WACK flop beside the B lane pulses `m_axi_wack` for one clock after the B handshake on the `m_axi` side.

3.4.2 Triple Buffer Design

The module buffers all three write channels independently:

1. **AW Channel Buffer**: Carries address, control, and `AWSNOOP`
2. **W Channel Buffer**: Carries data, strobes, and `WLAST`
3. **B Channel Buffer**: Carries write responses back to the FUB side

3.4.3 WACK Generation

The ACE write acknowledge is auto-pulsed by the module:

```
assign m_axi_wack = r_wack;
// r_wack <= m_axi_bvalid && m_axi_bready;
```

A one-cycle pulse is emitted one cycle after the write response handshake. The current subset does not rely on `WACK` for ordering.

3.4.4 Busy Signal Generation

```
assign busy = (int_aw_count > 0) || (int_w_count > 0) || (int_b_count
> 0) ||
                fub_axi_awvalid || fub_axi_wvalid || m_axi_bvalid;
```

3.5 ACE Notes

- `AWSNOOP[2:0]`, `AWDOMAIN[1:0]` and `AWBAR[1:0]` are straight payload pass-throughs; the module does not decode them or check that the combination is one Table D3-8 permits
- `AWUNIQUE` is not carried; it is needed only by a component that supports WriteEvict (IHI 0022H Table D2-2)
- `WACK` is generated locally; do not drive `m_axi_wack` from outside
- IDs (`AWID`/`BID`) are carried end-to-end; the module performs no ID matching
- Write data beats are not reordered; they pass through the W skid buffer in order

3.6 Related Modules

- **axi4ace_master_wr_monlite**: Lite-monitor wrapper for this core
- **axi4ace_slave_wr**: Complementary ACE write slave
- **axi4_master_wr**: AXI4-only base module
- **gaxi_skid_buffer**: Underlying buffer infrastructure

3.7 References

- Arm. *AMBA AXI and ACE Protocol Specification*, ARM IHI 0022H.c, Jan. 2021, Part D “AMBA ACE and ACE-Lite Protocol Specification”. developer.arm.com/documentation/ihi0022/h – §D2.1 “Changes to existing AXI channels” (`AWSNOOP`, `AWDOMAIN`, `AWBAR`), §D3.1 Tables D3-2, D3-5 and D3-8 (domain and barrier encodings and their permitted combinations), §D3.4 write response channel signaling, §D3.5 “Write Acknowledge signaling” (`WACK`). The optional `AWUNIQUE` (required only by a component that supports WriteEvict, Table D2-2) is not carried. The base valid/ready handshake is Part A §A3.2. Citation only.
- Carloni, L.P., McMillan, K.L., Sangiovanni-Vincentelli, A.L. “Theory of Latency-Insensitive Design.” *IEEE Trans. Computer-Aided Design of Integrated Circuits and Systems*, 20(9), Sept. 2001. DOI: 10.1109/43.945302 – the valid/ready channel discipline behind the `gaxi_skid_buffer` queues: a buffered stage may stall or add latency without changing what the endpoints compute.
- Maaref, M. *Architecting and Building High-Speed SoCs*, Packt, 2022, ch. 3 “Basic and Advanced On-Chip Busses and Interconnects”, sections “ARM AMBA interconnect protocols suite” (ACE) and “Data sharing and coherency challenges” – textbook treatment (secondary source).
- Indexed in `rtl/amba/references/README.md` (local PDF: `LatencyInsensitiveDesign__Carloni2001.pdf`)

3.8 Navigation

- ← Back to ACE Index
- ← Back to rtl-amba Index
- ← Back to Main Documentation Index

4 axi4ace_slave_rd

An AXI4 + ACE read slave module that buffers AR and R channels while carrying the ACE ARSNOOP[3:0], ARDOMAIN[1:0] and ARBAR[1:0] address fields from the slave interface to the backend, the 4-bit ACE RRESP back, and the manager's RACK through to the backend.

4.1 Overview

The axi4ace_slave_rd module is the ACE extension of axi4_slave_rd. It receives AXI4 read requests with the ACE ARSNOOP[3:0], ARDOMAIN[1:0] and ARBAR[1:0] fields on the s_axi_* interface, buffers them, and presents them on the fub_axi_* backend interface. Read data returns with the 4-bit ACE RRESP, and the RACK the manager drives on s_axi_rack is forwarded to fub_axi_rack. It is a transport module: it does not generate read data or interpret snoop encodings.

4.2 Parameters

Parameter	Type	Default	Description
SKID_DEPTH_AR	int	2	Address channel skid buffer depth in entries (2..8 inclusive)
SKID_DEPTH_R	int	4	Read data channel skid buffer depth in entries (2..8 inclusive)
AXI_ID_WIDTH	int	8	AXI transaction ID width
AXI_ADDR_WIDTH	int	32	AXI address bus width
AXI_DATA_WIDTH	int	32	AXI data bus width
AXI_USER_WIDTH	int	1	AXI user signal width
AXI_WSTRB_WIDTH	int	AXI_DATA_WIDTH/8	Write strobe width (unused, kept for naming consistency)
AW	int	AXI_ADDR_WIDTH	Short alias for address width
DW	int	AXI_DATA_WIDTH	Short alias for data width
IW	int	AXI_ID_WIDTH	Short alias for ID width
SW	int	AXI_WSTRB_WIDTH	Short alias for strobe width
UW	int	AXI_USER_WIDTH	Short alias for user width
ARSize	int	IW+AW+8+3+2+1+4+3+4+4+UW+4+2+2	Packed AR skid payload width
RSize	int	IW+DW+4+1+UW	Packed R skid payload width

4.3 Ports

4.3.1 Module Declaration

```
module axi4ace_slave_rd #(
    parameter int SKID_DEPTH_AR     = 2,
    parameter int SKID_DEPTH_R      = 4,

    parameter int AXI_ID_WIDTH      = 8,
    parameter int AXI_ADDR_WIDTH    = 32,
    parameter int AXI_DATA_WIDTH    = 32,
    parameter int AXI_USER_WIDTH    = 1,
    parameter int AXI_WSTRB_WIDTH   = AXI_DATA_WIDTH / 8,
    // Short params and calculations
    parameter int AW       = AXI_ADDR_WIDTH,
    parameter int DW       = AXI_DATA_WIDTH,
    parameter int IW       = AXI_ID_WIDTH,
    /* verilator lint_off UNUSEDPARAM */
    parameter int SW       = AXI_WSTRB_WIDTH,
    /* verilator lint_on UNUSEDPARAM */
    parameter int UW       = AXI_USER_WIDTH,
    parameter int ARSize   = IW+AW+8+3+2+1+4+3+4+4+UW+4+2+2,
    parameter int RSize    = IW+DW+4+1+UW
)
(
    // Global Clock and Reset
    input  logic aclk,
    input  logic aresetn,

    // Slave AXI Interface (Input Side)
    // Read address channel (AR)
    input  logic [IW-1:0]               s_axi_arid,
    input  logic [AW-1:0]               s_axi_araddr,
    input  logic [7:0]                  s_axi_arlen,
    input  logic [2:0]                  s_axi_arsize,
    input  logic [1:0]                  s_axi_arburst,
    input  logic                        s_axi_arlock,
    input  logic [3:0]                  s_axi_arcache,
    input  logic [2:0]                  s_axi_arprot,
    input  logic [3:0]                  s_axi_arqos,
    input  logic [3:0]                  s_axi_arregion,
    input  logic [UW-1:0]               s_axi_aruser,
    input  logic [3:0]                  s_axi_arsnoop,
    input  logic [1:0]                  s_axi_ardomain,
    input  logic [1:0]                  s_axi_arbar,
    input  logic                        s_axi_arvalid,
    output logic                        s_axi_arready,
```

```
    // Read data channel (R)
    output logic [IW-1:0]               s_axi_rid,
    output logic [DW-1:0]               s_axi_rdata,
    output logic [3:0]                  s_axi_rresp,      //
[3]=IsShared [2]=PassDirty (IHI 0022H D2.1.3)
    output logic                        s_axi_rlast,
    output logic [UW-1:0]               s_axi_ruser,
    output logic                        s_axi_rvalid,
    input  logic                        s_axi_rready,

    // ACE read acknowledge (IHI 0022H D2.3.1): driven by the manager
after
    // it accepts each RLAST beat, passed through to the backend
    input  logic                        s_axi_rack,

    // Master AXI Interface (Output Side to memory or backend)
    // Read address channel (AR)
    output logic [IW-1:0]               fub_axi_arid,
    output logic [AW-1:0]               fub_axi_araddr,
    output logic [7:0]                  fub_axi_arlen,
    output logic [2:0]                  fub_axi_arsize,
    output logic [1:0]                  fub_axi_arburst,
    output logic                        fub_axi_arlock,
    output logic [3:0]                  fub_axi_arcache,
    output logic [2:0]                  fub_axi_arprot,
    output logic [3:0]                  fub_axi_arqos,
    output logic [3:0]                  fub_axi_arregion,
    output logic [UW-1:0]               fub_axi_aruser,
    output logic [3:0]                  fub_axi_arsnoop,
    output logic [1:0]                  fub_axi_ardomain,
    output logic [1:0]                  fub_axi_arbar,
    output logic                        fub_axi_arvalid,
    input  logic                        fub_axi_arready,

    // Read data channel (R)
    input  logic [IW-1:0]               fub_axi_rid,
    input  logic [DW-1:0]               fub_axi_rdata,
    input  logic [3:0]                  fub_axi_rresp,      //
[3]=IsShared [2]=PassDirty (IHI 0022H D2.1.3)
    input  logic                        fub_axi_rlast,
    input  logic [UW-1:0]               fub_axi_ruser,
    input  logic                        fub_axi_rvalid,
    output logic                        fub_axi_rready,

    // ACE read acknowledge, forwarded from s_axi_rack
    output logic                        fub_axi_rack,
```

```
    // Status outputs for clock gating
    output logic                            busy
);
```

4.3.2 Clock and Reset

Port	Width	Direction	Description
aclk	1	Input	AXI clock
aresetn	1	Input	AXI active-low reset

4.3.3 Slave AXI Interface (Input Side)

4.3.3.1 Read Address Channel (AR)

Port	Width	Direction	Description
s_axi_arid	AXI_ID_WIDTH	Input	Read address ID
s_axi_araddr	AXI_ADDR_WIDTH	Input	Read address
s_axi_arlen	8	Input	Burst length, AXI-encoded: beats - 1
s_axi_arsize	3	Input	Transfer size (bytes per beat)
s_axi_arburst	2	Input	Burst type (FIXED, INCR, WRAP)
s_axi_arlock	1	Input	Lock type (atomic access)
s_axi_arcache	4	Input	Cache attributes
s_axi_arprot	3	Input	Protection attributes
s_axi_arqos	4	Input	Quality of Service
s_axi_arregion	4	Input	Region identifier
s_axi_arus	AXI_USER_W	Input	User-defined signals

Port	Width	Direction	Description
er	IDTH		
s_axi_arsnoop	4	Input	ACE read transaction type
s_axi_ardomain	2	Input	ACE shareability domain (ARDOMAIN): 00 Non-shareable, 01 Inner, 10 Outer, 11 System
s_axi_arbar	2	Input	ACE barrier type (ARBAR): 00 normal, 01 memory barrier, 10 normal ignoring barriers, 11 synchronization barrier
s_axi_arvalid	1	Input	Read address valid
s_axi_arready	1	Output	Read address ready

4.3.3.2 Read Data Channel (R)

Port	Width	Direction	Description
s_axi_rid	AXI_ID_WIDTH	Output	Read data ID
s_axi_rdata	AXI_DATA_WIDTH	Output	Read data
s_axi_rresp	4	Output	ACE read response: [1:0] OKAY/EXOKAY/SLVERR/DECERR, [2] PassDirty, [3] IsShared
s_axi_rlast	1	Output	Last read transfer in burst
s_axi_ruser	AXI_USER_WIDTH	Output	User-defined signals
s_axi_rvalid	1	Output	Read data valid
s_axi_rready	1	Input	Read data ready

4.3.4 Master AXI Interface (Output Side - FUB)

4.3.4.1 Read Address Channel (AR)

Port	Width	Direction	Description
fub_axi_arid	AXI_ID_WIDTH	Output	Read address ID
fub_axi_araddr	AXI_ADDR_WIDTH	Output	Read address
fub_axi_arlen	8	Output	Burst length, AXI-encoded: beats - 1
fub_axi_arsize	3	Output	Transfer size (bytes per beat)
fub_axi_arburst	2	Output	Burst type (FIXED, INCR, WRAP)
fub_axi_arlock	1	Output	Lock type (atomic access)
fub_axi_arcache	4	Output	Cache attributes
fub_axi_arprot	3	Output	Protection attributes
fub_axi_arqos	4	Output	Quality of Service
fub_axi_arregion	4	Output	Region identifier
fub_axi_aruser	AXI_USER_WIDTH	Output	User-defined signals
fub_axi_arsnoop	4	Output	ACE read transaction type
fub_axi_ardomain	2	Output	ACE shareability domain (ARDOMAIN): 00 Non-shareable, 01 Inner, 10 Outer, 11 System
fub_axi_arbar	2	Output	ACE barrier type (ARBAR): 00 normal, 01 memory barrier, 10 normal

Port	Width	Direction	Description
			ignoring barriers, 11 synchronization barrier
fub_axi_arvalid	1	Output	Read address valid
fub_axi_arready	1	Input	Read address ready

4.3.4.2 Read Data Channel (R)

Port	Width	Direction	Description
fub_axi_rid	AXI_ID_WIDTH	Input	Read data ID
fub_axi_rdata	AXI_DATA_WIDTH	Input	Read data
fub_axi_rresp	4	Input	ACE read response: [1:0] OKAY/EXOKAY/SLVERR/DECERR, [2] PassDirty, [3] IsShared
fub_axi_rlast	1	Input	Last read transfer in burst
fub_axi_ruser	AXI_USER_WIDTH	Input	User-defined signals
fub_axi_rvalid	1	Input	Read data valid
fub_axi_rready	1	Output	Read data ready

4.3.5 ACE Acknowledge

Port	Width	Direction	Description
s_axi_rack	1	Input	RACK from the upstream manager: a one-cycle pulse after the last read data beat (RLAST) handshake
fub_axi_ra	1	Output	`s_axi_rack` forwarded to

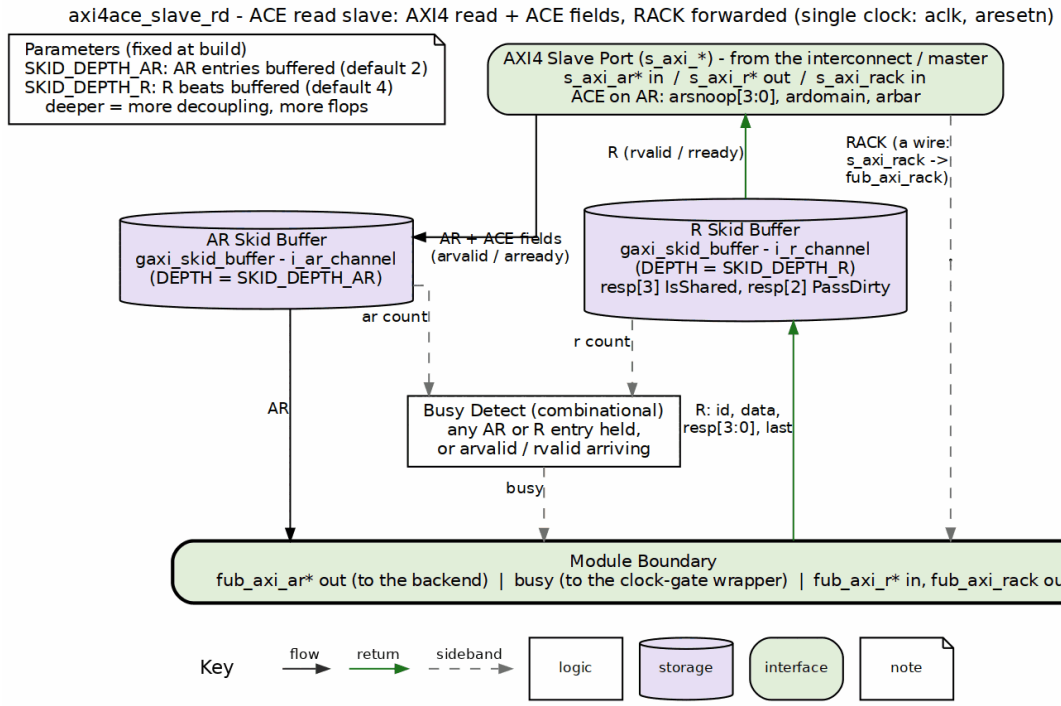
Port	Width	Direction	Description
ck			the backend unchanged

4.3.6 Status Interface

Port	Width	Direction	Description
busy	1	Output	Module activity indicator for clock gating

4.4 Functional Description

4.4.1 Block diagram



axi4ace_slave_rd block diagram

The axi4 read slave's layout (see `axi4_slave_rd`) plus what ACE adds. AR, with `arsnoop`, `ardomain` and `arbar`, drops down the left through its skid buffer to the backend. R, with the 4-bit `rresp` (IsShared, PassDirty), climbs the right. RACK has no payload and no handshake: it is a wire from `s_axi_rack` to `fub_axi_rack`, drawn dashed down the far right.

4.4.2 Dual Buffer Design

The module buffers AR from the slave side to the backend and R from the backend back to the slave side:

1. **AR Channel Buffer**: Receives `s_axi_ar*` and presents `fub_axi_ar*`, including `ARSNOOP`, `ARDOMAIN` and `ARBAR`
2. **R Channel Buffer**: Receives `fub_axi_r*` and presents `s_axi_r*`, including all four `RRESP` bits

4.4.3 RACK Forwarding

```
assign fub_axi_rack = s_axi_rack;
```

RACK has no payload and no handshake of its own, so it is a wire. The manager raises it only after the RLAST beat has left the R skid, so the backend always sees it after it handed that beat over (IHI 0022H §D3.3).

4.4.4 Busy Signal Generation

```
assign busy = (int_ar_count > 0) || (int_r_count > 0) ||
                s_axi_arvalid || fub_axi_rvalid;
```

4.5 ACE Notes

- ARSNOOP[3:0], ARDOMAIN[1:0] and ARBAR[1:0] are carried straight through from s_axi_ar* to fub_axi_ar*
- RRESP[3:2] (IsShared, PassDirty) is carried from fub_axi_rresp to s_axi_rresp unchanged
- The module does not generate RACK; the upstream manager drives s_axi_rack, and the module forwards it to fub_axi_rack
- IDs (ARID/RID) are carried end-to-end; the module performs no ID matching
- AXI_WSTRB_WIDTH is present only for naming consistency with the write slave and is unused

4.6 Related Modules

- **axi4ace_slave_rd_monlite**: Lite-monitor wrapper for this core
- **axi4ace_master_rd**: Complementary ACE read master
- **axi4_slave_rd**: AXI4-only base module
- **gaxi_skid_buffer**: Underlying buffer infrastructure

4.7 References

- Arm. *AMBA AXI and ACE Protocol Specification*, ARM IHI 0022H.c, Jan. 2021, Part D “AMBA ACE and ACE-Lite Protocol Specification”. developer.arm.com/documentation/ihi0022/h – §D2.1 “Changes to existing AXI channels” (`ARSNOOP`, `ARDOMAIN`, `ARBAR`, the 4-bit `RRESP`), §D3.1 Tables D3-2, D3-5 and D3-7 (domain and barrier encodings and their permitted combinations), §D3.2 read data channel signaling (Table D3-14, `RRESP[3:2]` = IsShared, PassDirty), §D3.3 “Read acknowledge signaling” (`RACK`). The base valid/ready handshake is Part A §A3.2. Citation only.
- Carloni, L.P., McMillan, K.L., Sangiovanni-Vincentelli, A.L. “Theory of Latency-Insensitive Design.” *IEEE Trans. Computer-Aided Design of Integrated Circuits and Systems*, 20(9), Sept. 2001. DOI: 10.1109/43.945302 – the valid/ready channel discipline behind the `gaxi_skid_buffer` queues: a buffered stage may stall or add latency without changing what the endpoints compute.
- Maaref, M. *Architecting and Building High-Speed SoCs*, Packt, 2022, ch. 3 “Basic and Advanced On-Chip Busses and Interconnects”, sections “ARM AMBA interconnect protocols suite” (ACE) and “Data sharing and coherency challenges” – textbook treatment (secondary source).
- Indexed in `rtl/amba/references/README.md` (local PDF: `LatencyInsensitiveDesign__Carloni2001.pdf`)

4.8 Navigation

- ← Back to ACE Index
- ← Back to rtl-amba Index
- ← Back to Main Documentation Index

5 axi4ace_slave_wr

An AXI4 + ACE write slave module that buffers AW, W, and B channels while carrying the ACE `AWSNOOP[2:0]`, `AWDOMAIN[1:0]` and `AWBAR[1:0]` address fields from the slave interface to the backend, and the manager's `WACK` through to the backend.

5.1 Overview

The axi4ace_slave_wr module is the ACE extension of axi4_slave_wr. It receives AXI4 write requests with the ACE AWSNOOP[2:0], AWDOMAIN[1:0] and AWBAR[1:0] fields on the s_axi_* interface, buffers them, and presents them on the fub_axi_* backend interface. Responses from the backend are buffered back to the slave interface, and the WACK the manager drives on s_axi_wack is forwarded to fub_axi_wack. It is a transport module and does not interpret snoop encodings.

5.2 Parameters

Parameter	Type	Default	Description
SKID_DEPTH_AW	int	2	Write address channel skid buffer depth in entries (2..8 inclusive)
SKID_DEPTH_W	int	4	Write data channel skid buffer depth in entries (2..8 inclusive)
SKID_DEPTH_B	int	2	Write response channel skid buffer depth in entries (2..8 inclusive)
AXI_ID_WIDTH	int	8	AXI transaction ID width
AXI_ADDR_WIDTH	int	32	AXI address bus width
AXI_DATA_WIDTH	int	32	AXI data bus width
AXI_USER_WIDTH	int	1	AXI user signal width
AXI_WSTRB_WIDTH	int	AXI_DATA_WIDTH/8	AXI write strobe width (calculated)
AW	int	AXI_ADDR_WIDTH	Short alias for address width
DW	int	AXI_DATA_WIDTH	Short alias for data width
IW	int	AXI_ID_WIDTH	Short alias for ID width
SW	int	AXI_WSTRB_WIDTH	Short alias for strobe width
UW	int	AXI_USER_WIDTH	Short alias for user width
AWSize	int	IW+AW+8+3+2+1+4+3+4+4+UW+3+2+2	Packed AW skid payload width
WSize	int	DW+SW+1+UW	Packed W skid payload width
BSize	int	IW+2+UW	Packed B skid payload width

5.3 Ports

5.3.1 Module Declaration

```
module axi4ace_slave_wr #(
    parameter int SKID_DEPTH_AW     = 2,
    parameter int SKID_DEPTH_W      = 4,
    parameter int SKID_DEPTH_B      = 2,
    parameter int AXI_ID_WIDTH      = 8,
    parameter int AXI_ADDR_WIDTH    = 32,
    parameter int AXI_DATA_WIDTH    = 32,
    parameter int AXI_USER_WIDTH    = 1,
    parameter int AXI_WSTRB_WIDTH   = AXI_DATA_WIDTH / 8,
    // Short params and calculations
    parameter int AW       = AXI_ADDR_WIDTH,
    parameter int DW       = AXI_DATA_WIDTH,
    parameter int IW       = AXI_ID_WIDTH,
    parameter int SW       = AXI_WSTRB_WIDTH,
    parameter int UW       = AXI_USER_WIDTH,
    parameter int AWSize   = IW+AW+8+3+2+1+4+3+4+4+UW+3+2+2,
    parameter int WSize    = DW+SW+1+UW,
    parameter int BSize    = IW+2+UW
)
(
    // Global Clock and Reset
    input  logic aclk,
    input  logic aresetn,

    // Slave AXI Interface (Input Side)
    // Write address channel (AW)
    input  logic [IW-1:0]               s_axi_awid,
    input  logic [AW-1:0]               s_axi_awaddr,
    input  logic [7:0]                  s_axi_awlen,
    input  logic [2:0]                  s_axi_awsize,
    input  logic [1:0]                  s_axi_awburst,
    input  logic                        s_axi_awlock,
    input  logic [3:0]                  s_axi_awcache,
    input  logic [2:0]                  s_axi_awprot,
    input  logic [3:0]                  s_axi_awqos,
    input  logic [3:0]                  s_axi_awregion,
    input  logic [UW-1:0]               s_axi_awuser,
    input  logic [2:0]                  s_axi_awsnoop,
    input  logic [1:0]                  s_axi_awdomain,
    input  logic [1:0]                  s_axi_awbar,
    input  logic                        s_axi_awvalid,
    output logic                        s_axi_awready,
```

```
    // Write data channel (W)
    input  logic [DW-1:0]               s_axi_wdata,
    input  logic [SW-1:0]               s_axi_wstrb,
    input  logic                        s_axi_wlast,
    input  logic [UW-1:0]               s_axi_wuser,
    input  logic                        s_axi_wvalid,
    output logic                        s_axi_wready,

    // Write response channel (B)
    output logic [IW-1:0]               s_axi_bid,
    output logic [1:0]                  s_axi_bresp,
    output logic [UW-1:0]               s_axi_buser,
    output logic                        s_axi_bvalid,
    input  logic                        s_axi_bready,

    // ACE write acknowledge (IHI 0022H D2.3.2): driven by the manager
after
    // it accepts each B response, passed through to the backend
    input  logic                        s_axi_wack,

    // Master AXI Interface (Output Side to memory or backend)
    // Write address channel (AW)
    output logic [IW-1:0]              fub_axi_awid,
    output logic [AW-1:0]              fub_axi_awaddr,
    output logic [7:0]                 fub_axi_awlen,
    output logic [2:0]                 fub_axi_awsize,
    output logic [1:0]                 fub_axi_awburst,
    output logic                       fub_axi_awlock,
    output logic [3:0]                 fub_axi_awcache,
    output logic [2:0]                 fub_axi_awprot,
    output logic [3:0]                 fub_axi_awqos,
    output logic [3:0]                 fub_axi_awregion,
    output logic [UW-1:0]              fub_axi_awuser,
    output logic [2:0]                 fub_axi_awsnoop,
    output logic [1:0]                 fub_axi_awdomain,
    output logic [1:0]                 fub_axi_awbar,
    output logic                       fub_axi_awvalid,
    input  logic                       fub_axi_awready,

    // Write data channel (W)
    output logic [DW-1:0]              fub_axi_wdata,
    output logic [SW-1:0]              fub_axi_wstrb,
    output logic                       fub_axi_wlast,
    output logic [UW-1:0]              fub_axi_wuser,
    output logic                       fub_axi_wvalid,
    input  logic                       fub_axi_wready,
```

```
    // Write response channel (B)
    input  logic [IW-1:0]               fub_axi_bid,
    input  logic [1:0]                  fub_axi_bresp,
    input  logic [UW-1:0]               fub_axi_buser,
    input  logic                        fub_axi_bvalid,
    output logic                        fub_axi_bready,

    // ACE write acknowledge, forwarded from s_axi_wack
    output logic                        fub_axi_wack,

    // Status outputs for clock gating
    output logic                        busy
);
```

5.3.2 Clock and Reset

Port	Width	Direction	Description
aclk	1	Input	AXI clock
aresetn	1	Input	AXI active-low reset

5.3.3 Slave AXI Interface (Input Side)

5.3.3.1 Write Address Channel (AW)

Port	Width	Direction	Description
s_axi_awid	AXI_ID_WIDTH	Input	Write address ID
s_axi_awaddr	AXI_ADDR_WIDTH	Input	Write address
s_axi_awlen	8	Input	Burst length, AXI-encoded: beats - 1
s_axi_awsize	3	Input	Transfer size (bytes per beat)
s_axi_awburst	2	Input	Burst type (FIXED, INCR, WRAP)
s_axi_awlock	1	Input	Lock type (atomic access)
s_axi_awcache	4	Input	Cache attributes
s_axi_awpr	3	Input	Protection attributes

Port	Width	Direction	Description
ot			
s_axi_awqos	4	Input	Quality of Service
s_axi_awregion	4	Input	Region identifier
s_axi_awuser	AXI_USER_WIDTH	Input	User-defined signals
s_axi_awsnoop	3	Input	ACE write transaction type
s_axi_awdomain	2	Input	ACE shareability domain (AWDOMAIN): 00 Non-shareable, 01 Inner, 10 Outer, 11 System
s_axi_awbar	2	Input	ACE barrier type (AWBAR): 00 normal, 01 memory barrier, 10 normal ignoring barriers, 11 synchronization barrier
s_axi_awvalid	1	Input	Write address valid
s_axi_awready	1	Output	Write address ready

5.3.3.2 Write Data Channel (W)

Port	Width	Direction	Description
s_axi_wdata	AXI_DATA_WIDTH	Input	Write data
s_axi_wstrb	AXI_WSTRB_WIDTH	Input	Write strobes
s_axi_wlast	1	Input	Last write transfer in burst
s_axi_wuser	AXI_USER_WID	Input	User-defined

Port	Width	Direction	Description
	TH		signals
s_axi_wvalid	1	Input	Write data valid
s_axi_wready	1	Output	Write data ready

5.3.3.3 Write Response Channel (B)

Port	Width	Direction	Description
s_axi_bid	AXI_ID_WIDTH	Output	Write response ID
s_axi_bresp	2	Output	Write response (OKAY, EXOKAY, SLVERR, DECERR)
s_axi_buser	AXI_USER_WIDTH	Output	User-defined signals
s_axi_bvalid	1	Output	Write response valid
s_axi_bready	1	Input	Write response ready

5.3.4 Master AXI Interface (Output Side - FUB)

5.3.4.1 Write Address Channel (AW)

Port	Width	Direction	Description
fub_axi_awid	AXI_ID_WIDTH	Output	Write address ID
fub_axi_awaddr	AXI_ADDR_WIDTH	Output	Write address
fub_axi_awlen	8	Output	Burst length, AXI-encoded: beats - 1
fub_axi_awsize	3	Output	Transfer size (bytes per beat)
fub_axi_awburst	2	Output	Burst type (FIXED, INCR, WRAP)
fub_axi_aw	1	Output	Lock type (atomic access)

Port	Width	Direction	Description
lock			
fub_axi_awcache	4	Output	Cache attributes
fub_axi_awprot	3	Output	Protection attributes
fub_axi_awqos	4	Output	Quality of Service
fub_axi_awregion	4	Output	Region identifier
fub_axi_awuser	AXI_USER_WIDTH	Output	User-defined signals
fub_axi_awsnoop	3	Output	ACE write transaction type
fub_axi_awdomain	2	Output	ACE shareability domain (AWDOMAIN): 00 Non-shareable, 01 Inner, 10 Outer, 11 System
fub_axi_awbar	2	Output	ACE barrier type (AWBAR): 00 normal, 01 memory barrier, 10 normal ignoring barriers, 11 synchronization barrier
fub_axi_awvalid	1	Output	Write address valid
fub_axi_awready	1	Input	Write address ready

5.3.4.2 Write Data Channel (W)

Port	Width	Direction	Description
fub_axi_wdata	AXI_DATA_WIDTH	Output	Write data
fub_axi_wstrb	AXI_WSTRB_WIDTH	Output	Write strobes

Port	Width	Direction	Description
fub_axi_wlast	1	Output	Last write transfer in burst
fub_axi_wuser	AXI_USER_WIDTH	Output	User-defined signals
fub_axi_wvalid	1	Output	Write data valid
fub_axi_wready	1	Input	Write data ready

5.3.4.3 Write Response Channel (B)

Port	Width	Direction	Description
fub_axi_bid	AXI_ID_WIDTH	Input	Write response ID
fub_axi_bresp	2	Input	Write response (OKAY, EXOKAY, SLVERR, DECERR)
fub_axi_buser	AXI_USER_WIDTH	Input	User-defined signals
fub_axi_bvalid	1	Input	Write response valid
fub_axi_bready	1	Output	Write response ready

5.3.5 ACE Acknowledge

Port	Width	Direction	Description
s_axi_wack	1	Input	WACK from the upstream manager: a one-cycle pulse after the write response (B) handshake
fub_axi_wack	1	Output	`s_axi_wack` forwarded to the backend unchanged

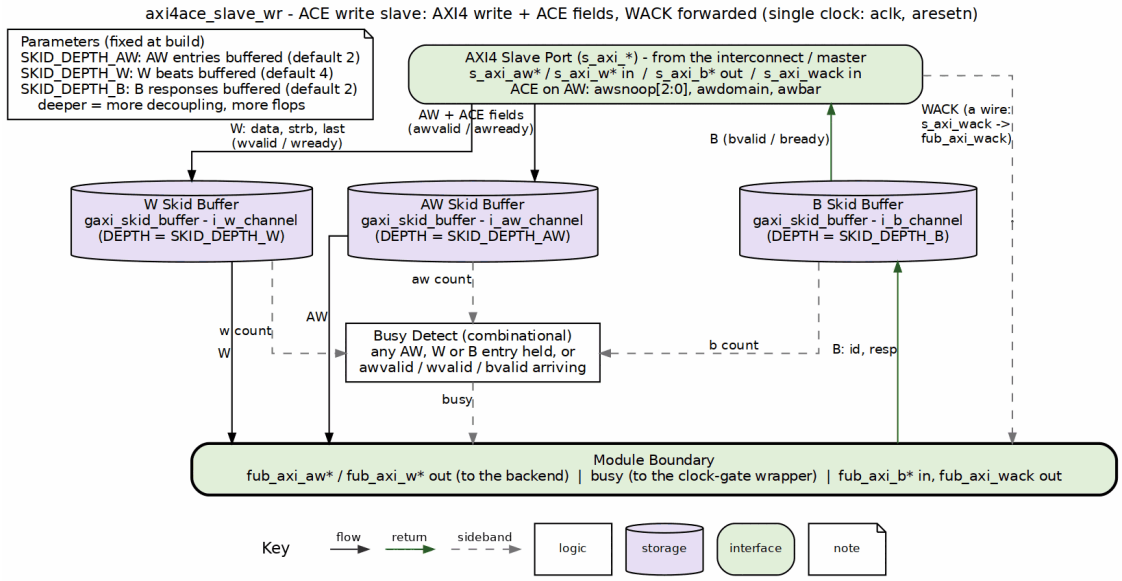
5.3.6 Status Interface

Port	Width	Direction	Description
busy	1	Output	Module activity indicator for clock gating

Open Source - MIT License

5.4 Functional Description

5.4.1 Block diagram



axi4ace_slave_wr block diagram

The axi4 write slave's layout (see `axi4_slave_wr`) plus what ACE adds. W and AW, with `awsnoop`, `awdomain` and `awbar`, drop down through their skid buffers to the backend, and B climbs the right lane. WACK is a wire from `s_axi_wack` to `fub_axi_wack`, drawn dashed down the far right.

5.4.2 Triple Buffer Design

The module buffers all three write channels:

1. **AW Channel Buffer**: Receives `s_axi_aw*` and presents `fub_axi_aw*`, including `AWSNOOP`
2. **W Channel Buffer**: Receives `s_axi_w*` and presents `fub_axi_w*`
3. **B Channel Buffer**: Receives `fub_axi_b*` and presents `s_axi_b*`

5.4.3 Busy Signal Generation

```
assign busy = (int_aw_count > 0) || (int_w_count > 0) || (int_b_count
> 0) ||
                  s_axi_awvalid || s_axi_wvalid || fub_axi_bvalid;
```

5.5 ACE Notes

- `AWSNOOP[2:0]`, `AWDOMAIN[1:0]` and `AWBAR[1:0]` are carried straight through from `s_axi_aw*` to `fub_axi_aw*`
- The module does not generate `WACK`; the upstream manager drives `s_axi_wack`, and the module forwards it to `fub_axi_wack` as a wire (`assign fub_axi_wack = s_axi_wack;`). The manager raises it only after the B response has left the B skid, so the backend sees it after it handed that response over (IHI 0022H §D3.5)
- `AWUNIQUE` is not carried; it is needed only by a component that supports WriteEvict (IHI 0022H Table D2-2)
- IDs (`AWID`/`BID`) are carried end-to-end; the module performs no ID matching
- Write data beats are not reordered; they pass through the W skid buffer in order

5.6 Related Modules

- **axi4ace_slave_wr_monlite**: Lite-monitor wrapper for this core
- **axi4ace_master_wr**: Complementary ACE write master
- **axi4_slave_wr**: AXI4-only base module
- **gaxi_skid_buffer**: Underlying buffer infrastructure

5.7 References

- Arm. *AMBA AXI and ACE Protocol Specification*, ARM IHI 0022H.c, Jan. 2021, Part D “AMBA ACE and ACE-Lite Protocol Specification”. developer.arm.com/documentation/ihi0022/h – §D2.1 “Changes to existing AXI channels” (`AWSNOOP`, `AWDOMAIN`, `AWBAR`), §D3.1 Tables D3-2, D3-5 and D3-8 (domain and barrier encodings and their permitted combinations), §D3.4 write response channel signaling, §D3.5 “Write Acknowledge signaling” (`WACK`). The optional `AWUNIQUE` (required only by a component that supports WriteEvict, Table D2-2) is not carried. The base valid/ready handshake is Part A §A3.2. Citation only.
- Carloni, L.P., McMillan, K.L., Sangiovanni-Vincentelli, A.L. “Theory of Latency-Insensitive Design.” *IEEE Trans. Computer-Aided Design of Integrated Circuits and Systems*, 20(9), Sept. 2001. DOI: 10.1109/43.945302 – the valid/ready channel discipline behind the `gaxi_skid_buffer` queues: a buffered stage may stall or add latency without changing what the endpoints compute.
- Maaref, M. *Architecting and Building High-Speed SoCs*, Packt, 2022, ch. 3 “Basic and Advanced On-Chip Busses and Interconnects”, sections “ARM AMBA interconnect protocols suite” (ACE) and “Data sharing and coherency challenges” – textbook treatment (secondary source).
- Indexed in `rtl/amba/references/README.md` (local PDF: `LatencyInsensitiveDesign__Carloni2001.pdf`)

5.8 Navigation

- ← Back to ACE Index
- ← Back to rtl-amba Index
- ← Back to Main Documentation Index

6 axi4ace_snoop_master

ACE snoop-channel initiator transport on the coherency manager (CCU) side: AC out to a peer cache, CR/CD in from that cache.

6.1 Overview

The `axi4ace_snoop_master` module sits between the CCU logic (onyx) and a peer cache responder (amber/jet). It receives a snoop command from the CCU on the FUB-side AC channel, buffers it, and drives it to the cache on the `m_axi_*` interface. The cache's CR and CD responses are buffered back to the CCU. It is a transport module and does not generate snoop commands or interpret responses.

6.2 Parameters

Parameter	Type	Default	Description
SKID_DEPTH_AC	int	2	Snoop address channel skid buffer depth in entries (2..8 inclusive)
SKID_DEPTH_CR	int	4	Snoop response channel skid buffer depth in entries (2..8 inclusive)
SKID_DEPTH_CD	int	4	Snoop data channel skid buffer depth in entries (2..8 inclusive)
ADDR_WIDTH	int	32	Snoop address width
DATA_WIDTH	int	32	Snoop data width
AW	int	ADDR_WIDTH	Short alias for address width
DW	int	DATA_WIDTH	Short alias for data width
ACSize	int	AW + 4 + 3	Packed AC skid payload width
CRSize	int	5	Packed CR skid payload width
CDSize	int	DW + 1	Packed CD skid payload width

6.3 Ports

6.3.1 Module Declaration

```
module axi4ace_snoop_master #(
    parameter int SKID_DEPTH_AC = 2,
    parameter int SKID_DEPTH_CR = 4,
    parameter int SKID_DEPTH_CD = 4,
    parameter int ADDR_WIDTH    = 32,
    parameter int DATA_WIDTH    = 32,
    parameter int AW            = ADDR_WIDTH,
    parameter int DW            = DATA_WIDTH,
    parameter int ACSize        = AW + 4 + 3,
    parameter int CRSize        = 5,
    parameter int CDSize        = DW + 1
) (
    // Global Clock and Reset
    input  logic                        aclk,
    input  logic                        aresetn,

    // Slave AXI Interface (Input Side) -- upstream CCU logic
    // Snoop address channel (AC)
    input  logic [AW-1:0]               fub_acaddr,
    input  logic [3:0]                  fub_acsnoop,
    input  logic [2:0]                  fub_acprot,
    input  logic                        fub_acvalid,
    output logic                        fub_acready,

    // Snoop response channel (CR)
    output logic [4:0]                  fub_crresp,
    output logic                        fub_crvalid,
    input  logic                        fub_crready,

    // Snoop data channel (CD)
    output logic [DW-1:0]               fub_cddata,
    output logic                        fub_cdlast,
    output logic                        fub_cdvalid,
    input  logic                        fub_cdready,

    // Master AXI Interface (Output Side) -- peer cache responder
    // Snoop address channel (AC)
    output logic [AW-1:0]               m_axi_acaddr,
    output logic [3:0]                  m_axi_acsnoop,
    output logic [2:0]                  m_axi_acprot,
    output logic                        m_axi_acvalid,
    input  logic                        m_axi_acready,
```

```
    // Snoop response channel (CR)
    input  logic [4:0]                  m_axi_crresp,
    input  logic                        m_axi_crvalid,
    output logic                        m_axi_crready,

    // Snoop data channel (CD)
    input  logic [DW-1:0]               m_axi_cddata,
    input  logic                        m_axi_cdlast,
    input  logic                        m_axi_cdvalid,
    output logic                        m_axi_cdready,

    // Status outputs for clock gating
    output logic                        busy
);
```

6.3.2 Clock and Reset

Port	Width	Direction	Description
aclk	1	Input	AXI clock
aresetn	1	Input	AXI active-low reset

6.3.3 CCU Interface (Input Side - FUB)

6.3.3.1 Snoop Address Channel (AC)

Port	Width	Direction	Description
fub_acaddr	ADDR_WIDTH	Input	Snoop address
fub_acsnoop	4	Input	Snoop transaction type
fub_acprot	3	Input	Protection attributes
fub_acvalid	1	Input	Snoop address valid
fub_acready	1	Output	Snoop address ready

6.3.3.2 Snoop Response Channel (CR)

Port	Width	Direction	Description
fub_crresp	5	Output	Snoop

Port	Width	Direction	Description
			response
fub_crvalid	1	Output	Snoop response valid
fub_crready	1	Input	Snoop response ready

6.3.3.3 Snoop Data Channel (CD)

Port	Width	Direction	Description
fub_cddata	DATA_WIDTH	Output	Snoop data
fub_cdlast	1	Output	Last snoop data beat
fub_cdvalid	1	Output	Snoop data valid
fub_cdready	1	Input	Snoop data ready

6.3.4 Cache Responder Interface (Output Side - AXI)

6.3.4.1 Snoop Address Channel (AC)

Port	Width	Direction	Description
m_axi_acaddr	ADDR_WIDTH	Output	Snoop address
m_axi_acsnoop	4	Output	Snoop transaction type
m_axi_acprot	3	Output	Protection attributes
m_axi_acvalid	1	Output	Snoop address valid
m_axi_acready	1	Input	Snoop address ready

6.3.4.2 Snoop Response Channel (CR)

Port	Width	Direction	Description
m_axi_crresp	5	Input	Snoop response

Port	Width	Direction	Description
m_axi_crvalid	1	Input	Snoop response valid
m_axi_crready	1	Output	Snoop response ready

6.3.4.3 Snoop Data Channel (CD)

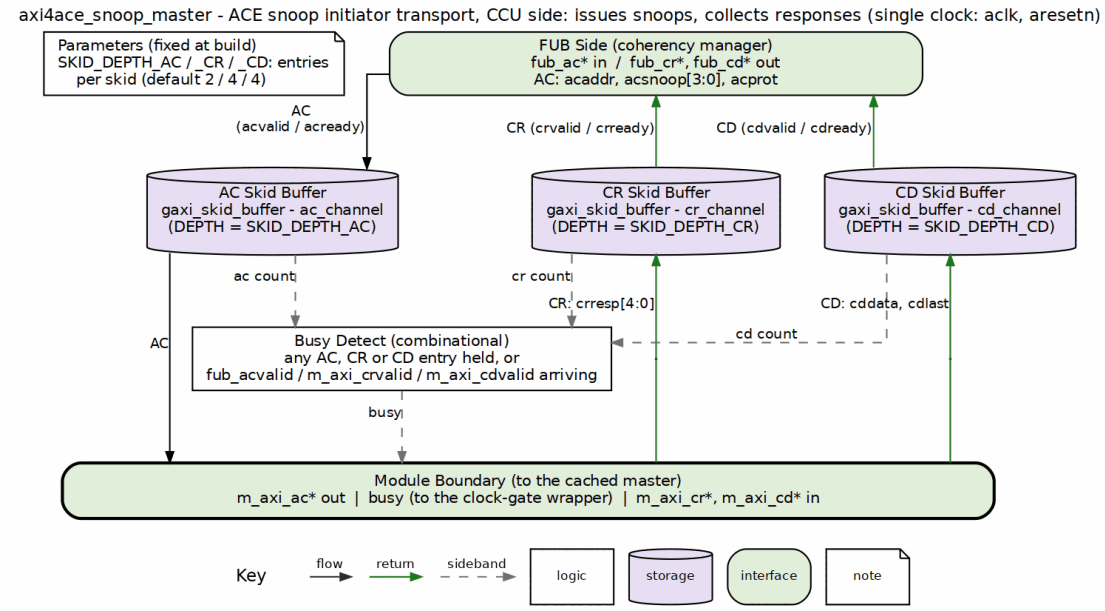
Port	Width	Direction	Description
m_axi_cddata	DATA_WIDTH	Input	Snoop data
m_axi_cdlast	1	Input	Last snoop data beat
m_axi_cdvalid	1	Input	Snoop data valid
m_axi_cdready	1	Output	Snoop data ready

6.3.5 Status Interface

Port	Width	Direction	Description
busy	1	Output	Module activity indicator for clock gating

6.4 Functional Description

6.4.1 Block diagram



axi4ace_snoop_master block diagram

The snoop-channel initiator, on the CCU / coherency-manager side. The snoop request (AC: `acaddr`, `acsnoop`, `acprot`) drops down the left lane through its skid buffer to the cached master. The snoop response (CR, `crresp[4:0]`) and snoop data (CD, `cddata`, `cdlast`) climb their own lanes on the right through their own skid buffers. It is pure transport: no snoop decode and no ordering logic.

6.4.2 Triple Buffer Design

The module buffers all three snoop channels independently:

1. **AC Channel Buffer**: CCU → peer cache
2. **CR Channel Buffer**: Peer cache → CCU
3. **CD Channel Buffer**: Peer cache → CCU

6.4.3 Busy Signal Generation

```
assign busy = (int_ac_count > 0) || (int_cr_count > 0) ||
(int_cd_count > 0) ||
              fub_acvalid || m_axi_crvalid || m_axi_cdvalid;
```

6.5 ACE Notes

- Snoop channels have no transaction ID; ordering is by channel only
- ACE requires CR and CD responses after the AC handshake, and both CR and CD must be returned in the same order as the AC addresses
- The module is a transport; it does not arbitrate or serialize snoops across multiple peer caches
- `CRRESP` bit semantics follow ACE: DataTransfer[0], Error[1], PassDirty[2], IsShared[3], WasUnique[4]

6.6 Related Modules

- **axi4ace_snoop_master_monlite**: Lite-monitor wrapper for this core
- **axi4ace_snoop_slave**: Complementary cache-side snoop responder
- **axi4ace_snoop_monitor_lite**: ACE-aware monitor core for the snoop channels
- **gaxi_skid_buffer**: Underlying buffer infrastructure

6.7 References

- Arm. *AMBA AXI and ACE Protocol Specification*, ARM IHI 0022H.c, Jan. 2021, Part D “AMBA ACE and ACE-Lite Protocol Specification”. developer.arm.com/documentation/ihi0022/h – §D2.2 “Additional channels defined by ACE”, §D3.6 snoop address (AC), §D3.7 snoop response (CR, `CRRESP`), §D3.8 snoop data (CD) channel signaling, §D3.9 “Snoop channel dependencies”, ch. D5 “Snoop Transactions”. The base valid/ready handshake is Part A §A3.2. Citation only.
- Carloni, L.P., McMillan, K.L., Sangiovanni-Vincentelli, A.L. “Theory of Latency-Insensitive Design.” *IEEE Trans. Computer-Aided Design of Integrated Circuits and Systems*, 20(9), Sept. 2001. DOI: 10.1109/43.945302 – the valid/ready channel discipline behind the `gaxi_skid_buffer` queues: a buffered stage may stall or add latency without changing what the endpoints compute.
- Papamarcos, M.S., Patel, J.H. “A Low-Overhead Coherence Solution for Multiprocessors with Private Cache Memories.” *Proc. 11th Int. Symp. Computer Architecture (ISCA)*, 1984, pp. 348-354. DOI: 10.1145/800015.808204 – the snoopy write-back protocol with an exclusive-clean state (MESI) that the snoop channels serve; background only, since this module holds no coherence state.
- Sweazey, P., Smith, A.J. “A Class of Compatible Cache Consistency Protocols and their Support by the IEEE Futurebus.” *Proc. 13th ISCA*, 1986; *ACM SIGARCH Computer Architecture News*, 14(2), May 1986, pp. 414-423. DOI: 10.1145/17356.17404 – the MOESI state class (the Owned state, `PassDirty` ownership transfer); background only.
- Nagarajan, V., Sorin, D.J., Hill, M.D., Wood, D.A. *A Primer on Memory Consistency and Cache Coherence*, 2nd ed. Synthesis Lectures on Computer Architecture, Morgan & Claypool, 2020. DOI: 10.1007/978-3-031-01764-3 – ch. 7 “Snooping Coherence Protocols” (§7.3 Exclusive state, §7.4 Owned state, §7.5 non-atomic bus): what a snoop request and response mean to the caches at each end.
- Maaref, M. *Architecting and Building High-Speed SoCs*, Packt, 2022, ch. 3 “Basic and Advanced On-Chip Busses and Interconnects”, sections “ARM AMBA interconnect protocols suite” (ACE) and “Data sharing and coherency challenges” – textbook treatment (secondary source).

- Handy, J. *The Cache Memory Book*, 2nd ed., 1998, ch. 4 “Maintaining Coherency in Cached Systems”, §4.3.1 “The MESI Protocol” and §4.3.3 “The MOESI Protocol” – textbook treatment (secondary source).
- Hennessy, J.L., Patterson, D.A. *Computer Architecture: A Quantitative Approach*, 6th ed., Morgan Kaufmann, 2019, ch. 5 “Thread-Level Parallelism”, §5.2 “Centralized Shared-Memory Architectures” – snooping coherence and the MESI/MOESI extensions (secondary source).
- Indexed in `rtl/amba/references/README.md` (local PDF: `LatencyInsensitiveDesign__Carloni2001.pdf`, `MoesiCoherenceClass__Sweazey1986.pdf`, `projects/components/cache-ip/References/Papamarcos_Patel_1984_ISCA_Snoopy_Coherence.pdf`, `projects/components/cache-ip/References/Sorin_Hill_Wood_2020_Primer_Memory_Consistency_Cache_Coherence_2ed.pdf`)

6.8 Navigation

- ← Back to ACE Index
- ← Back to rtl-amba Index
- ← Back to Main Documentation Index

7 axi4ace_snoop_slave

ACE snoop-channel responder transport on the cache side: AC in from the coherency manager, CR/CD out back to the manager.

7.1 Overview

The `axi4ace_snoop_slave` module sits between the coherency manager (onyx) and a cache's snoop FSM (amber/jet). It receives a snoop command on the AC channel, buffers it, and presents it to the cache FSM on the FUB side. The cache FSM returns its response on CR and any data on CD; those are buffered back to the manager. It is a transport module and does not generate snoop responses or data.

7.2 Parameters

Parameter	Type	Default	Description
SKID_DEPTH_AC	int	2	Snoop address channel skid buffer depth in entries (2..8 inclusive)
SKID_DEPTH_CR	int	4	Snoop response channel skid buffer depth in entries (2..8 inclusive)
SKID_DEPTH_CD	int	4	Snoop data channel skid buffer depth in entries (2..8 inclusive)
ADDR_WIDTH	int	32	Snoop address width
DATA_WIDTH	int	32	Snoop data width
AW	int	ADDR_WIDTH	Short alias for address width
DW	int	DATA_WIDTH	Short alias for data width
ACSize	int	AW + 4 + 3	Packed AC skid payload width
CRSize	int	5	Packed CR skid payload width
CDSize	int	DW + 1	Packed CD skid payload width

7.3 Ports

7.3.1 Module Declaration

```
module axi4ace_snoop_slave #(
    parameter int SKID_DEPTH_AC = 2,
    parameter int SKID_DEPTH_CR = 4,
    parameter int SKID_DEPTH_CD = 4,
    parameter int ADDR_WIDTH    = 32,
    parameter int DATA_WIDTH    = 32,
    parameter int AW            = ADDR_WIDTH,
    parameter int DW            = DATA_WIDTH,
    parameter int ACSize        = AW + 4 + 3,
    parameter int CRSize        = 5,
    parameter int CDSize        = DW + 1
) (
    // Global Clock and Reset
    input  logic                        aclk,
    input  logic                        aresetn,

    // Slave AXI Interface (Input Side) -- manager/coherency
controller -> cache
    // Snoop address channel (AC)
    input  logic [AW-1:0]               m_axi_acaddr,
    input  logic [3:0]                  m_axi_acsnoop,
    input  logic [2:0]                  m_axi_acprot,
    input  logic                        m_axi_acvalid,
    output logic                        m_axi_acready,

    // Snoop response channel (CR)
    output logic [4:0]                  m_axi_crresp,
    output logic                        m_axi_crvalid,
    input  logic                        m_axi_crready,

    // Snoop data channel (CD)
    output logic [DW-1:0]               m_axi_cddata,
    output logic                        m_axi_cdlast,
    output logic                        m_axi_cdvalid,
    input  logic                        m_axi_cdready,

    // Fabric/Upstream Buffer Interface (Output Side) -- cache snoop
FSM
    // Snoop address channel (AC)
    output logic [AW-1:0]               fub_acaddr,
    output logic [3:0]                  fub_acsnoop,
    output logic [2:0]                  fub_acprot,
```

```
    output logic                        fub_acvalid,
    input  logic                        fub_acready,

    // Snoop response channel (CR)
    input  logic [4:0]                  fub_crresp,
    input  logic                        fub_crvalid,
    output logic                        fub_crready,

    // Snoop data channel (CD)
    input  logic [DW-1:0]               fub_cddata,
    input  logic                        fub_cdlast,
    input  logic                        fub_cdvalid,
    output logic                        fub_cdready,

    // Status outputs for clock gating
    output logic                        busy
);
```

7.3.2 Clock and Reset

Port	Width	Direction	Description
aclk	1	Input	AXI clock
aresetn	1	Input	AXI active-low reset

7.3.3 Manager Interface (Input Side - AXI)

7.3.3.1 Snoop Address Channel (AC)

Port	Width	Direction	Description
m_axi_acaddr	ADDR_WIDTH	Input	Snoop address
m_axi_acsnoop	4	Input	Snoop transaction type
m_axi_acprot	3	Input	Protection attributes
m_axi_acvalid	1	Input	Snoop address valid
m_axi_acready	1	Output	Snoop address ready

7.3.3.2 Snoop Response Channel (CR)

Port	Width	Direction	Description
m_axi_crresp	5	Output	Snoop response
m_axi_crvalid	1	Output	Snoop response valid
m_axi_crready	1	Input	Snoop response ready

7.3.3.3 Snoop Data Channel (CD)

Port	Width	Direction	Description
m_axi_cddata	DATA_WIDTH	Output	Snoop data
m_axi_cdlast	1	Output	Last snoop data beat
m_axi_cdvalid	1	Output	Snoop data valid
m_axi_cdready	1	Input	Snoop data ready

7.3.4 Cache FSM Interface (Output Side - FUB)

7.3.4.1 Snoop Address Channel (AC)

Port	Width	Direction	Description
fub_acaddr	ADDR_WIDTH	Output	Snoop address
fub_acsnoop	4	Output	Snoop transaction type
fub_acprot	3	Output	Protection attributes
fub_acvalid	1	Output	Snoop address valid
fub_acready	1	Input	Snoop address ready

7.3.4.2 Snoop Response Channel (CR)

Port	Width	Direction	Description
fub_crresp	5	Input	Snoop response
fub_crvalid	1	Input	Snoop response valid
fub_crready	1	Output	Snoop response ready

7.3.4.3 Snoop Data Channel (CD)

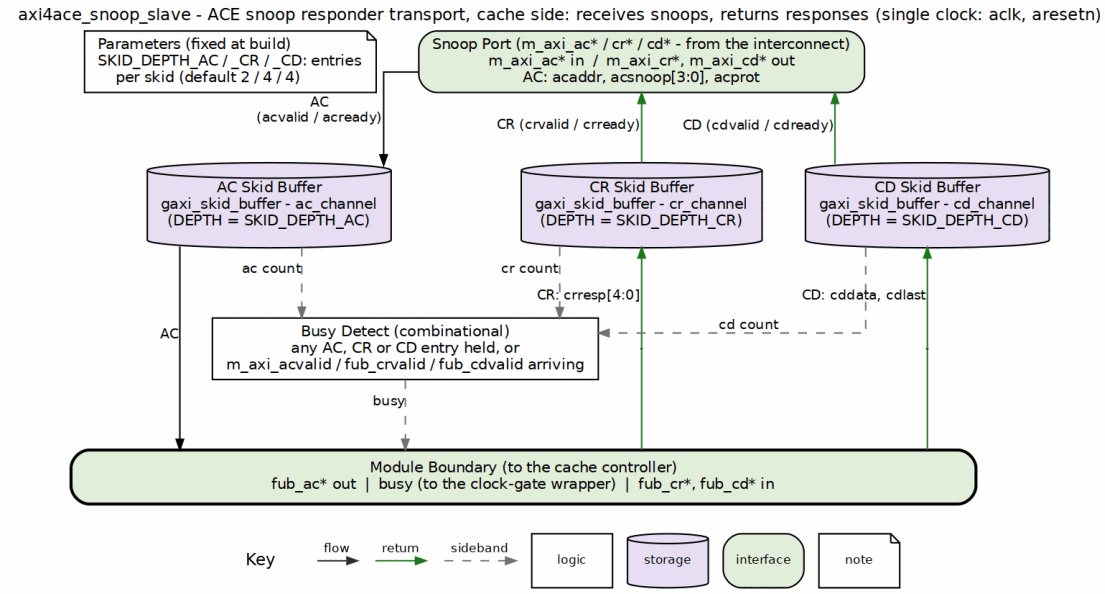
Port	Width	Direction	Description
fub_cddata	DATA_WIDTH	Input	Snoop data
fub_cdlast	1	Input	Last snoop data beat
fub_cdvalid	1	Input	Snoop data valid
fub_cdready	1	Output	Snoop data ready

7.3.5 Status Interface

Port	Width	Direction	Description
busy	1	Output	Module activity indicator for clock gating

7.4 Functional Description

7.4.1 Block diagram



axi4ace_snoop_slave block diagram

The snoop-channel responder, on the cache side and mirroring `axi4ace_snoop_master`. Snoop requests (AC) arrive from the interconnect and drop down the left lane through a skid buffer to the cache controller. The controller's response (CR) and data (CD) climb their own lanes on the right back to the interconnect. It is pure transport: no snoop decode and no ordering logic.

7.4.2 Triple Buffer Design

The module buffers all three snoop channels independently:

1. **AC Channel Buffer**: Manager → cache FSM
2. **CR Channel Buffer**: Cache FSM → manager
3. **CD Channel Buffer**: Cache FSM → manager

7.4.3 Busy Signal Generation

```
assign busy = (int_ac_count > 0) || (int_cr_count > 0) ||
(int_cd_count > 0) ||
              m_axi_acvalid || fub_crvalid || fub_cdvalid;
```

7.5 ACE Notes

- Snoop channels have no transaction ID; ordering is by channel only
- ACE requires CR and CD responses after the AC handshake, and both CR and CD must be returned in the same order as the AC addresses
- The module does not enforce the stricter `cache-ip` convention (complete CD before CR); that ordering is produced by the cache FSM driving the FUB side
- `CRRESP` bit semantics follow ACE: DataTransfer[0], Error[1], PassDirty[2], IsShared[3], WasUnique[4]

7.6 Related Modules

- **axi4ace_snoop_slave_monlite**: Lite-monitor wrapper for this core
- **axi4ace_snoop_master**: Complementary CCU-side snoop initiator
- **axi4ace_snoop_monitor_lite**: ACE-aware monitor core for the snoop channels
- **gaxi_skid_buffer**: Underlying buffer infrastructure

7.7 References

- Arm. *AMBA AXI and ACE Protocol Specification*, ARM IHI 0022H.c, Jan. 2021, Part D “AMBA ACE and ACE-Lite Protocol Specification”. developer.arm.com/documentation/ihi0022/h – §D2.2 “Additional channels defined by ACE”, §D3.6 snoop address (AC), §D3.7 snoop response (CR, `CRRESP`), §D3.8 snoop data (CD) channel signaling, §D3.9 “Snoop channel dependencies”, ch. D5 “Snoop Transactions”. The base valid/ready handshake is Part A §A3.2. Citation only.
- Carloni, L.P., McMillan, K.L., Sangiovanni-Vincentelli, A.L. “Theory of Latency-Insensitive Design.” *IEEE Trans. Computer-Aided Design of Integrated Circuits and Systems*, 20(9), Sept. 2001. DOI: 10.1109/43.945302 – the valid/ready channel discipline behind the `gaxi_skid_buffer` queues: a buffered stage may stall or add latency without changing what the endpoints compute.
- Papamarcos, M.S., Patel, J.H. “A Low-Overhead Coherence Solution for Multiprocessors with Private Cache Memories.” *Proc. 11th Int. Symp. Computer Architecture (ISCA)*, 1984, pp. 348-354. DOI: 10.1145/800015.808204 – the snoopy write-back protocol with an exclusive-clean state (MESI) that the snoop channels serve; background only, since this module holds no coherence state.
- Sweazey, P., Smith, A.J. “A Class of Compatible Cache Consistency Protocols and their Support by the IEEE Futurebus.” *Proc. 13th ISCA*, 1986; *ACM SIGARCH Computer Architecture News*, 14(2), May 1986, pp. 414-423. DOI: 10.1145/17356.17404 – the MOESI state class (the Owned state, `PassDirty` ownership transfer); background only.
- Nagarajan, V., Sorin, D.J., Hill, M.D., Wood, D.A. *A Primer on Memory Consistency and Cache Coherence*, 2nd ed. Synthesis Lectures on Computer Architecture, Morgan & Claypool, 2020. DOI: 10.1007/978-3-031-01764-3 – ch. 7 “Snooping Coherence Protocols” (§7.3 Exclusive state, §7.4 Owned state, §7.5 non-atomic bus): what a snoop request and response mean to the caches at each end.
- Maaref, M. *Architecting and Building High-Speed SoCs*, Packt, 2022, ch. 3 “Basic and Advanced On-Chip Busses and Interconnects”, sections “ARM AMBA interconnect protocols suite” (ACE) and “Data sharing and coherency challenges” – textbook treatment (secondary source).

- Handy, J. *The Cache Memory Book*, 2nd ed., 1998, ch. 4 “Maintaining Coherency in Cached Systems”, §4.3.1 “The MESI Protocol” and §4.3.3 “The MOESI Protocol” – textbook treatment (secondary source).
- Hennessy, J.L., Patterson, D.A. *Computer Architecture: A Quantitative Approach*, 6th ed., Morgan Kaufmann, 2019, ch. 5 “Thread-Level Parallelism”, §5.2 “Centralized Shared-Memory Architectures” – snooping coherence and the MESI/MOESI extensions (secondary source).
- Indexed in `rtl/amba/references/README.md` (local PDF: `LatencyInsensitiveDesign__Carloni2001.pdf`, `MoesiCoherenceClass__Sweazey1986.pdf`, `projects/components/cache-ip/References/Papamarcos_Patel_1984_ISCA_Snoopy_Coherence.pdf`, `projects/components/cache-ip/References/Sorin_Hill_Wood_2020_Primer_Memory_Consistency_Cache_Coherence_2ed.pdf`)

7.8 Navigation

- ← Back to ACE Index
- ← Back to rtl-amba Index
- ← Back to Main Documentation Index